BIG Data

Introduction

Thibault Giordan

- Data-Scientist depuis 2017 (Skapanê puis CleverConnect)
- GIS 2017 - Prez Poly'Games 2015-2016, Prez Rock 2016-2017
- Co-organisateur Meet-up Big Data et machine-learning
  - https://www.meetup.com/fr-FR/lille-big-data-and-machine-learning-meetup/
- Projet Text-mining/données non-structurées
- Projet Spark
- Adresse à privilégier : thibaultgiordan@gmail.com

Objectifs

- Découverte de Hadoop
- Manipulation de Spark
- Applications de méthodes de Text-mining
- Projet de Machine-learning/Text-mining
- Répondre à vos interrogations sur le Big Data

Plan du cours

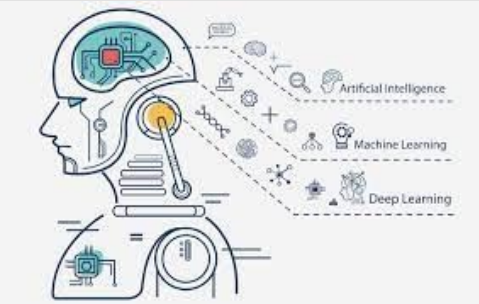
Date	Sujets	Notebooks/data	Prof
4h	Généralité sur Hadoop et Spark + manipulation basique sur spark + ML Spark	Notebook : 1 - Première analyse data : vgsales.csv	Thibault G.
4h	Machine-learning + début projet	offensive_comments.csv	Thibault G.
Reste du cours	Projet text-mining	offensive_comments.csv	Thibault L.

Big Data

Pourquoi ?
Qu’est ce que c’est ?

Machine-learning VS Big Data

- Machine-learning = apprentissage



- Big Data = Traitement de gros volumes

Les 3,4,5,6… V du Big Data

- Volume
- Vitesse
- Véracité
- Variété
- Valeur
- Visualisation
- …

Pourquoi l’IA maintenant

- Algorithme existe depuis 20-30 ans
- De plus en plus de données (via internet)
- Des machines plus puissantes
- Une société qui se digitalise

Dans la vie de tout les jours

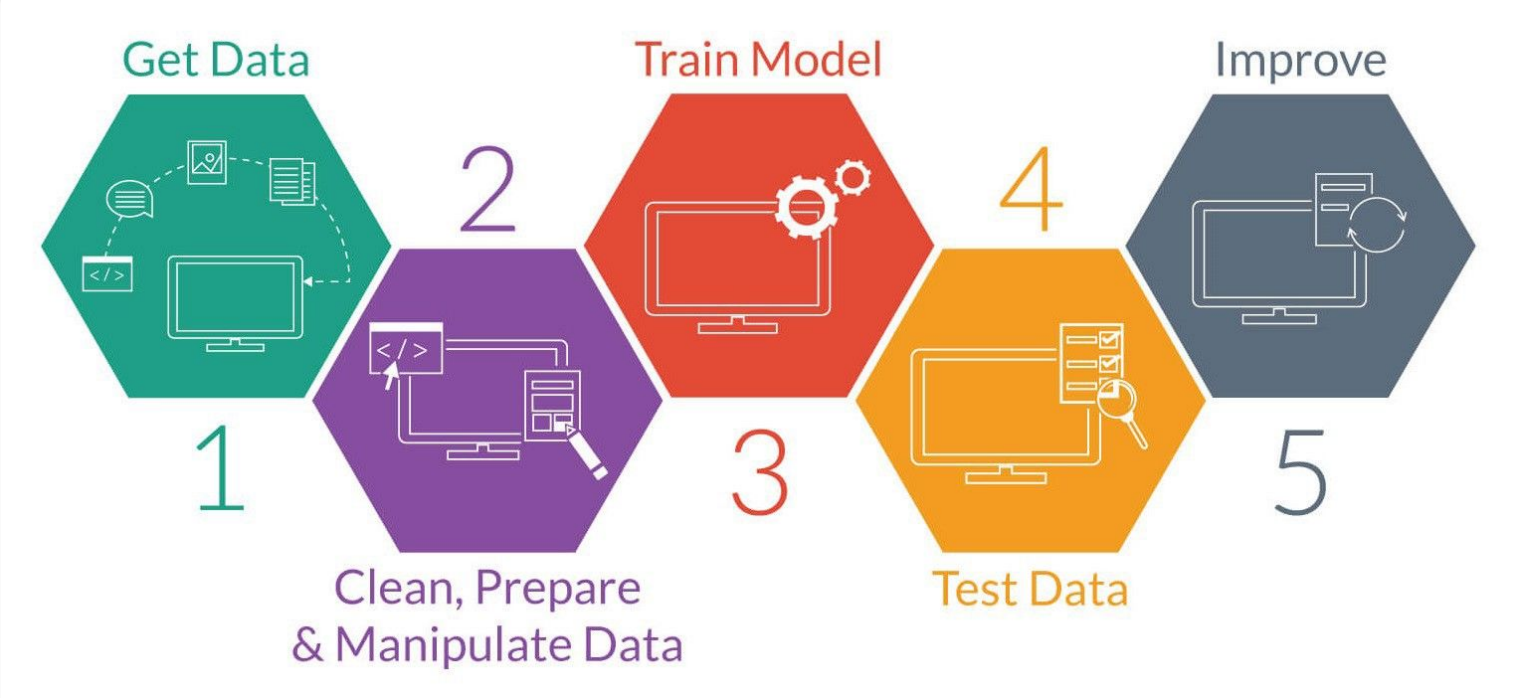
- Recommandation Netflix
- Chatbot
- Détection d’objets sur des photos (chien, chat, personnes etc...)
- Attribuer un crédit à une personne
- Classification de documents

Quel type de données ?

- De tout type : texte(CV, commentaire, article) , image, vidéo son
- Beaucoup de données
- Données historique qui sont “labelise” :

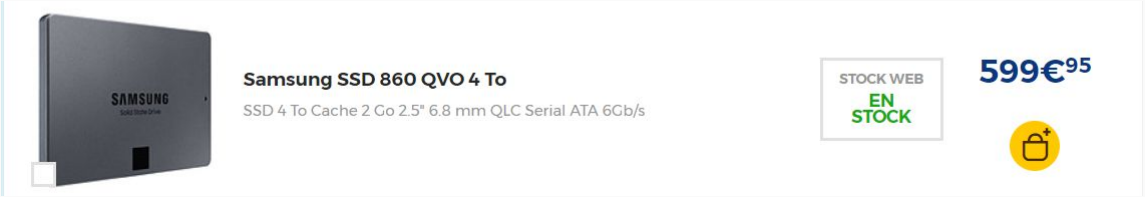
Exemple : 1 millions d’images ou on a noté les images de chien et les images qui ne sont pas des chiens

Étapes des projets data



Nécessite Big Data

- Stockage

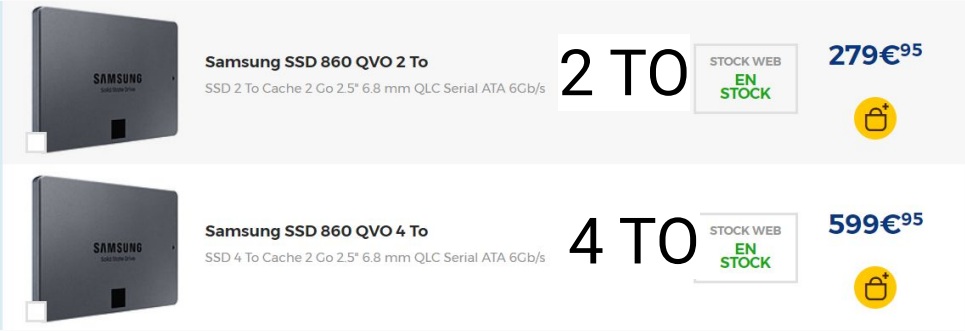


- RAM

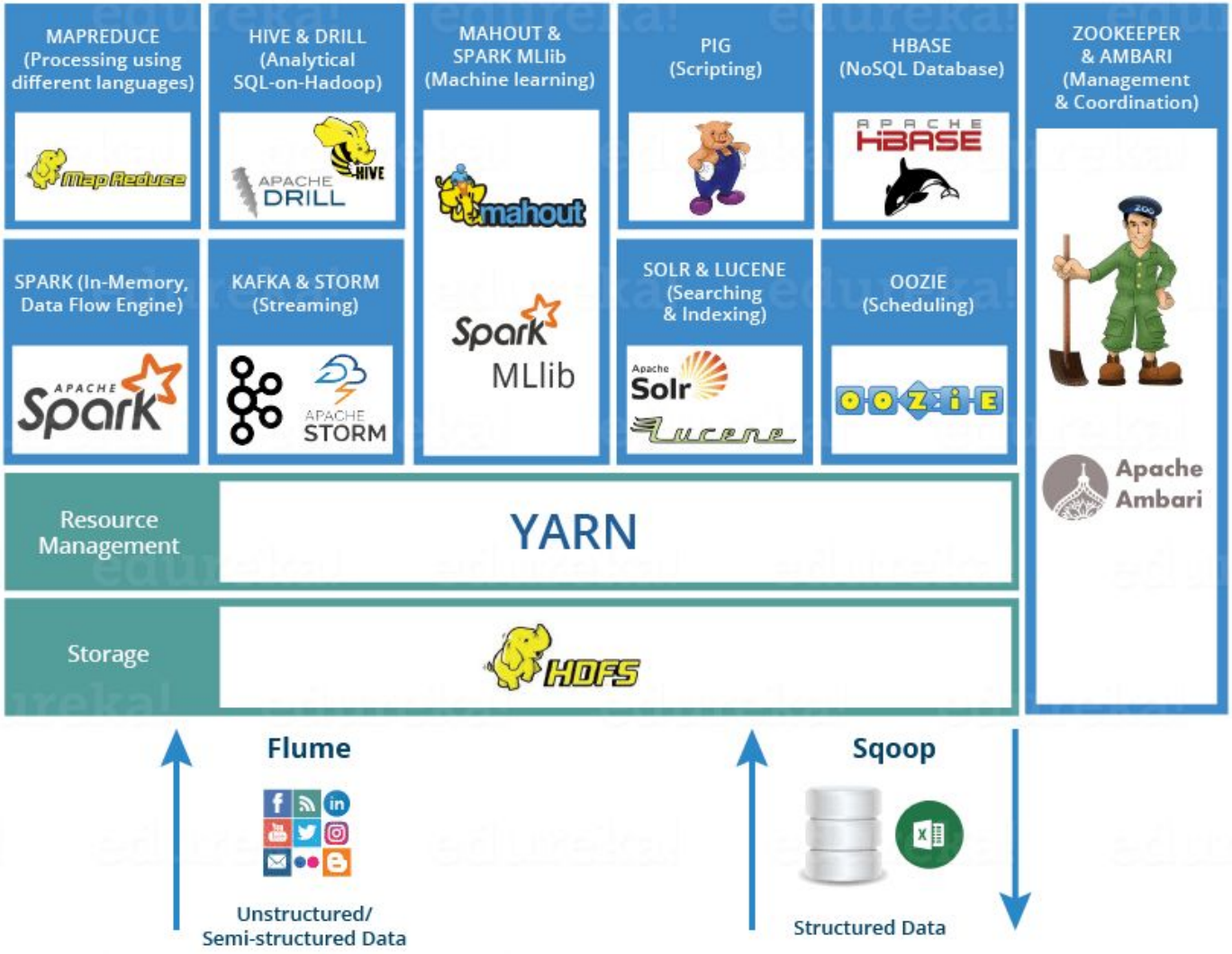
- Temps d’exécution

Principe : utiliser plusieurs machines

- Avantage
  - réduction des coûts d'achats
  - répartition des coups de remplacement
- Inconvénients
  - Outils particuliers



Outils Big Data



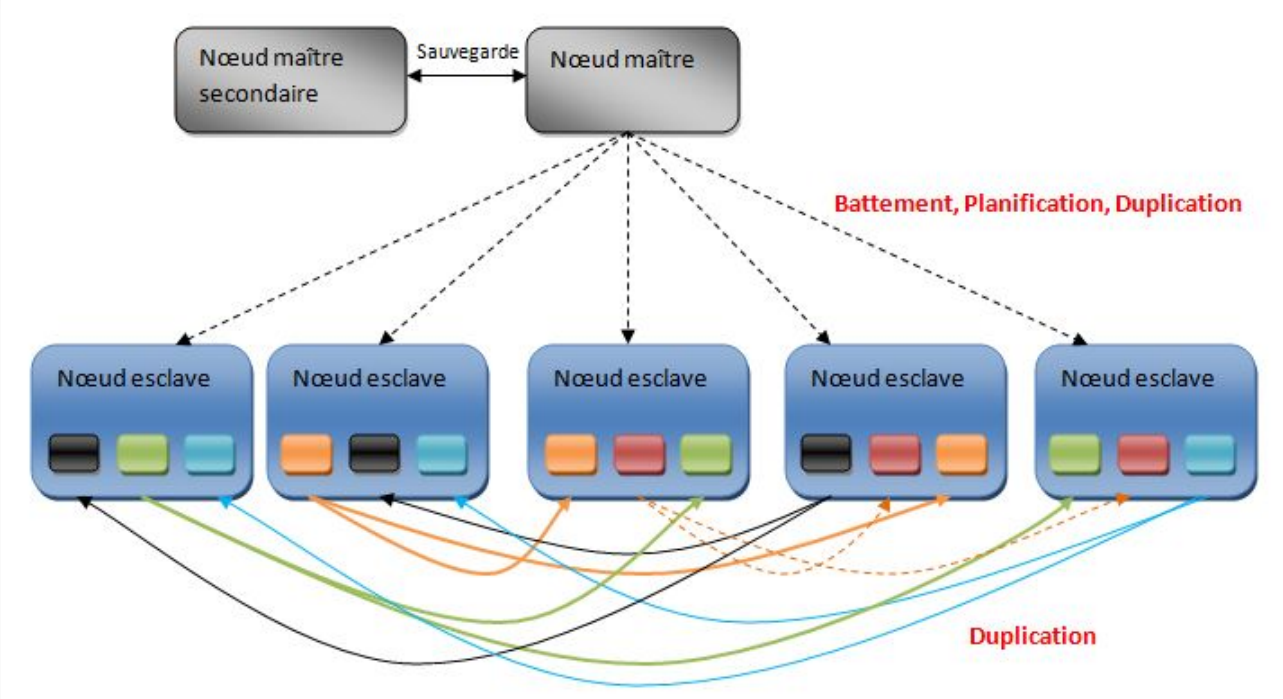
APACHE
hadoop
TM

Idée d’Hadoop

- Faciliter la création d’applications distribuées
- Scalable (fonctionne pour un cluster de 3,10,100, 2 000 machines)
- Traiter des pétaoctets de données
- Gestion automatique des pannes matérielles par le framework

HDFS = Hadoop distributed File system

Fonctionnement de hadoop HDFS

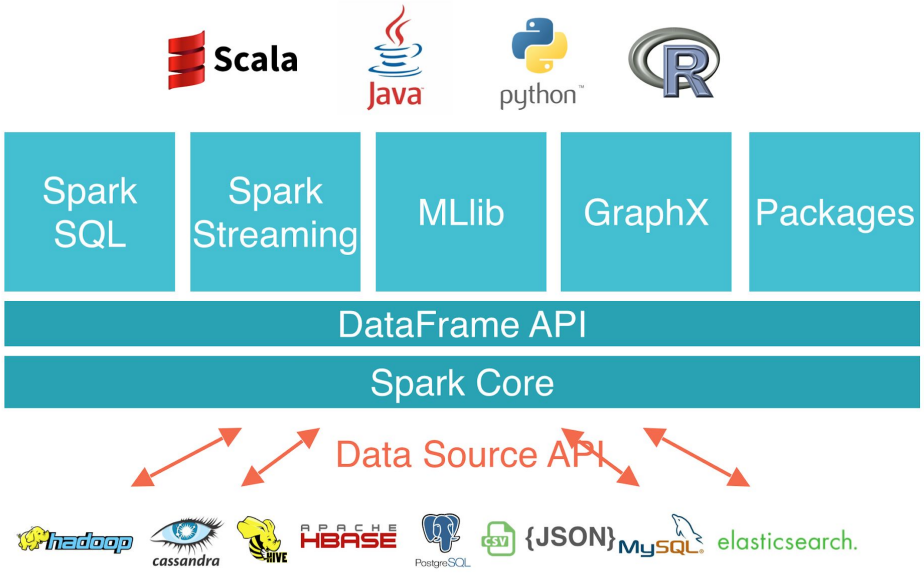


APACHE
Spark
TM

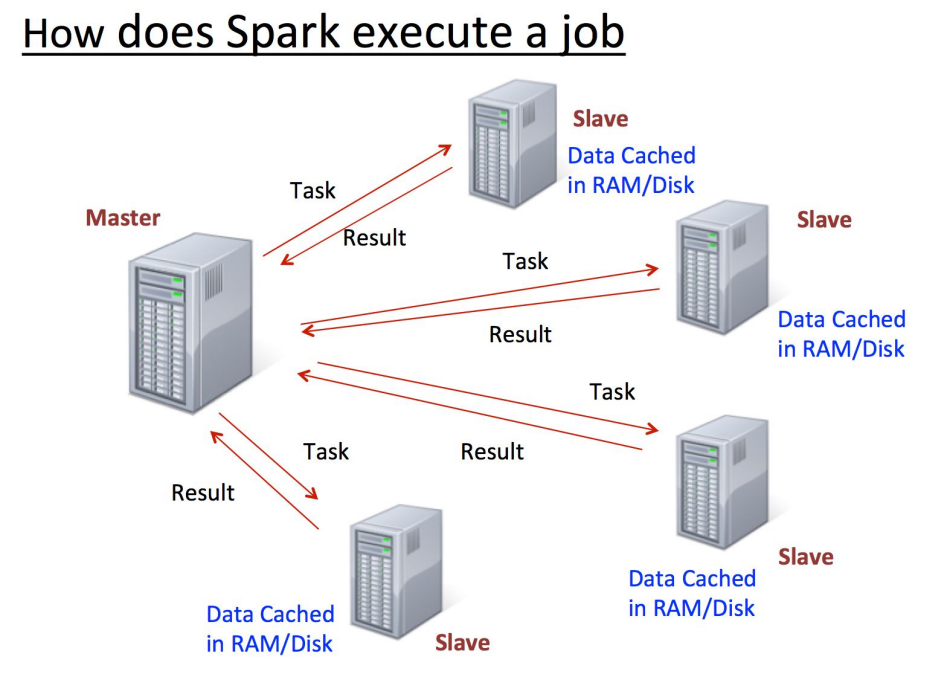
Idée de Spark

- Framework open source de calculs distribués
- Fonctionne avec de gros volumes de données
- Fonctionne avec le paradigme MapReduce

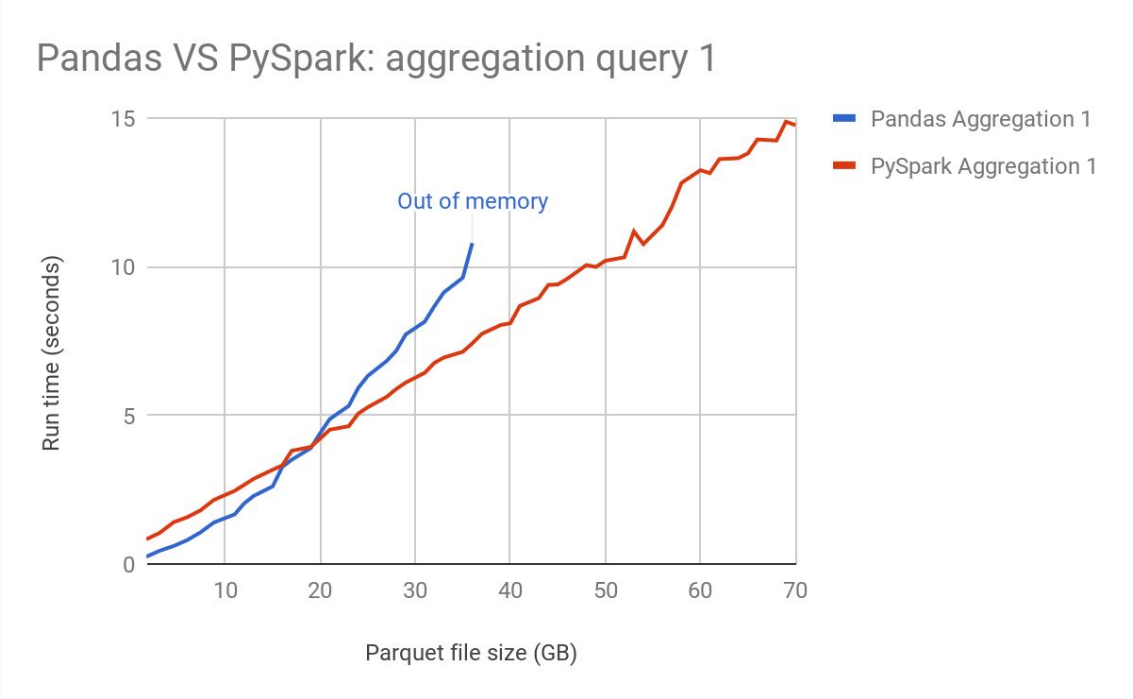
Présentation de Spark



Fonctionnement de Spark



Spark VS Pandas



MapReduce

Explication du Map

- Opération qui transforme n objets en m objets (m>=n), entrée <= sortie
  - Exemple 1 : Faire +1 à chaque âge sur chaque ligne
  - Exemple 2 : Transformer une phrase en plusieurs mots

Nom	Prenom	Age
Dupont	Bob	18
Martin	Louise	20

MAP →

Nom	Prenom	Age
Dupont	Bob	19
Martin	Louise	21

Explication du Reduce

- Opération qui transforme n objets en m objets (n>=m), sortie <= entrée
  - Exemple : Sommer des éléments

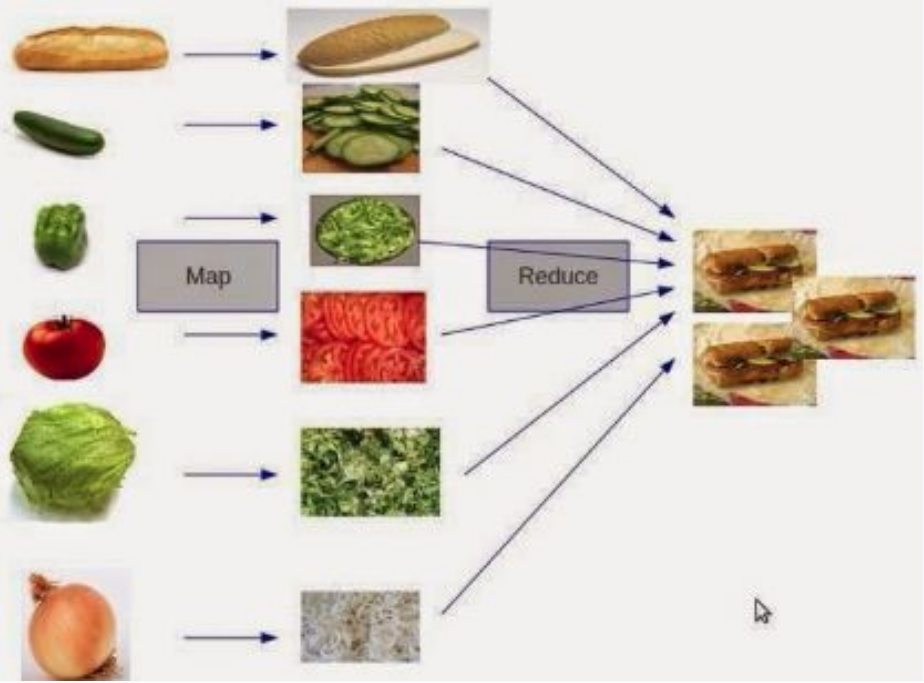
Nom	Prenom	Age
Dupont	Bob	18
Martin	Louise	20
		19
		10

Reduce

→ 38

→ 29

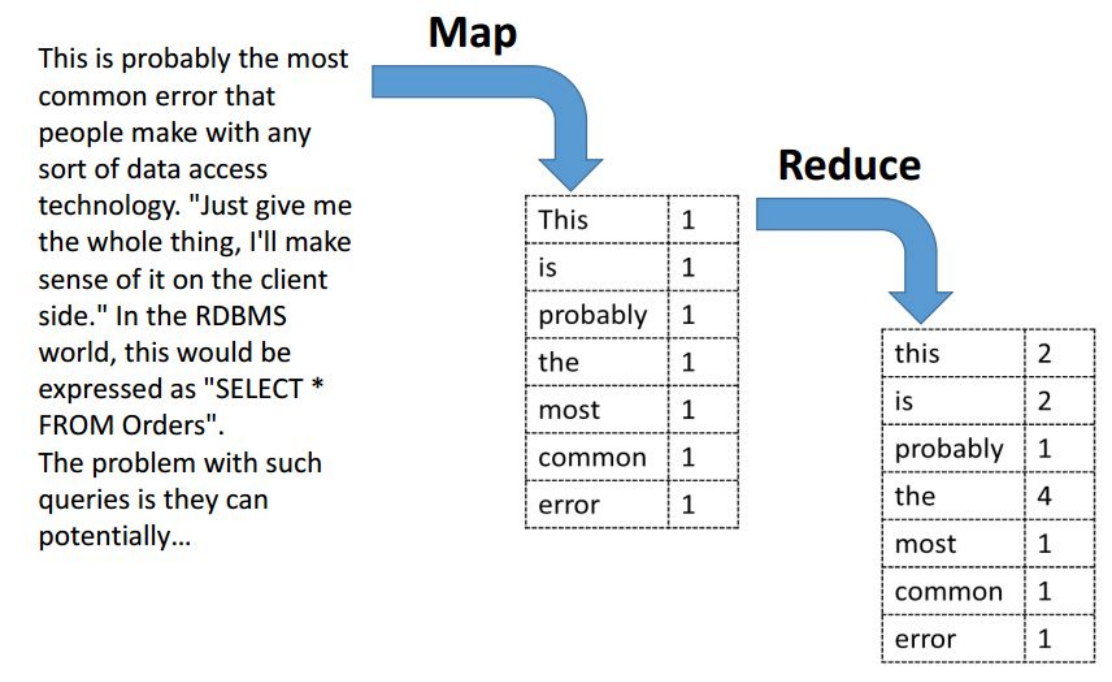
MapReduce exemple



Exemple

```
text_file= ("This is probably…",)

text_file
.flatMap(lambda line: line.split())
    .map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a+b)
```



Spark - Code

Structuration des données

Format	RDD (Resilient Distributed Dataset)	Dataframes
+	• Flexible • Plus de fonctionnalités bas niveaux • Les lignes peuvent être de tailles différentes	• Plus performant • Plus de fonctionnalités haut niveaux • Permet de faire du SQL • Librairie de Machine-Learning simple et en évolution
-	• Plus lent • Structure peu à peu abandonnée • Obligé de réécrire chaque ligne à chaque transformation	• Nécessite un typage • Nécessite que chaque ligne ait la même taille

Exemple de code

```
#Charger un csv
df = spark.read.csv(mon_path_csv)

#Filtrer sur les ages
df_majeur = df.where(df.age>=18)

#Passer d'un dataframe à un rdd et inversement
rdd_majeur = df_majeur.rdd
df_majeur_2 = rdd_majeur.toDF()
```

Install Spark

Utiliser Colab (à choisir en priorité)

- uploader les données et notebooks sur votre drive google
- (vous pouvez également uploader directement vos données sur le notebook)
- ajouter les lignes suivantes :
  - !pip install pyspark
  - from pyspark.sql import SparkSession
  - spark = SparkSession.builder.master("local[*]").getOrCreate()
  - sc = spark.sparkContext
  - from google.colab import drive
  - drive.mount('/My_drive/')

Utiliser Databricks (backup)

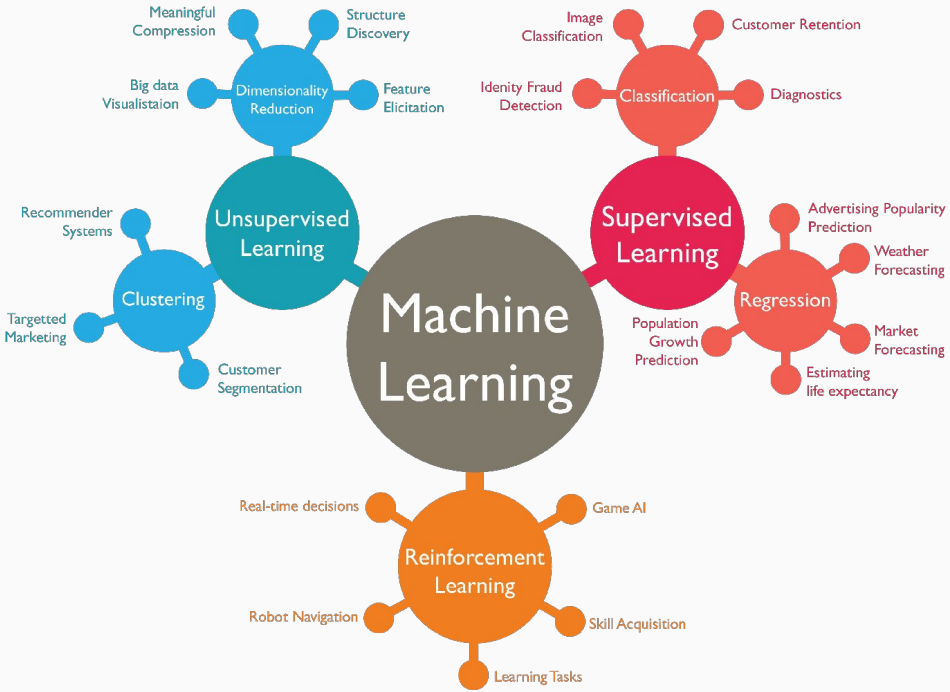
- https://community.cloud.databricks.com
- créer un cluster (laisser les paramètres par défaut)
- Uploader le csv “vgsales.csv” dans la partie “data”
- Uploader le Notebook “1 - Première analyse.ipynb” dans la partie “workspace”

Machine-learning

Introduction

- L'apprentissage supervisé (*supervised learning* en anglais) : création d'un modèle à partir de données labellisées. Le modèle va prédire la classe/le score des individus

- L'apprentissage non supervisé (*unsupervised learning* en anglais) : création d'un modèle à partir de données non labellisées. Le modèle va regrouper les individus similaires

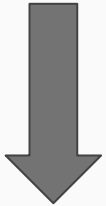


Classification

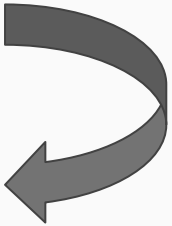
Features : [0;..][1;...][46;....]

Label : 1;2;1

Algorithme



Modèle

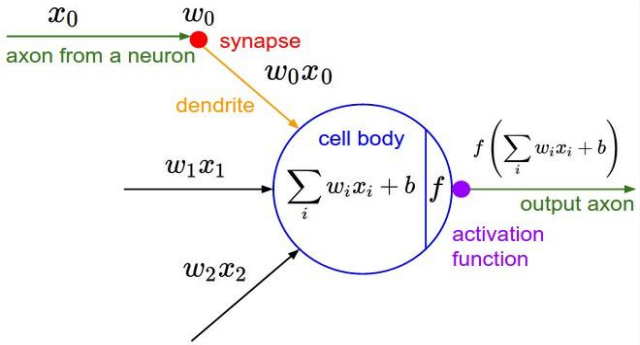


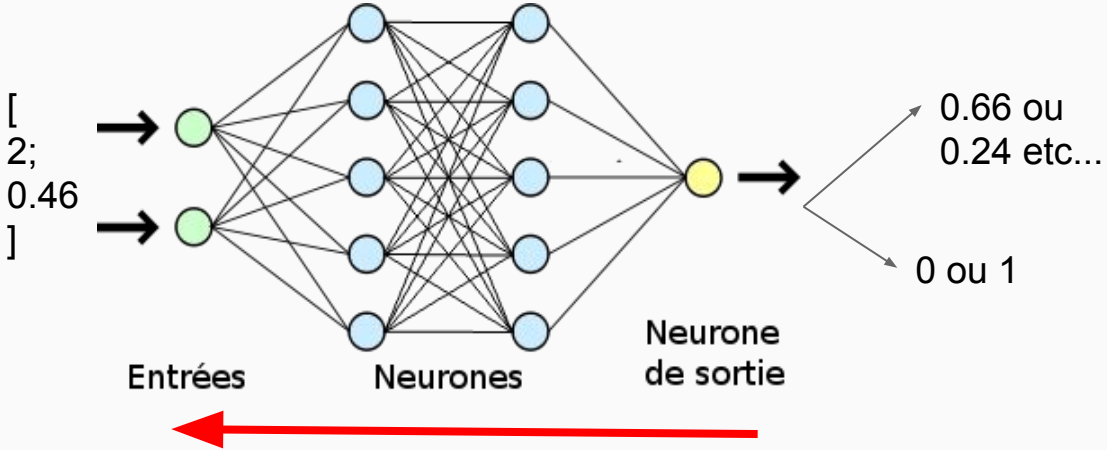
IN : [0 ; 1 ; 2 ; 45 ; 0.2 ; 0.8]

OUT : 1

Machine-learning (slide algo)

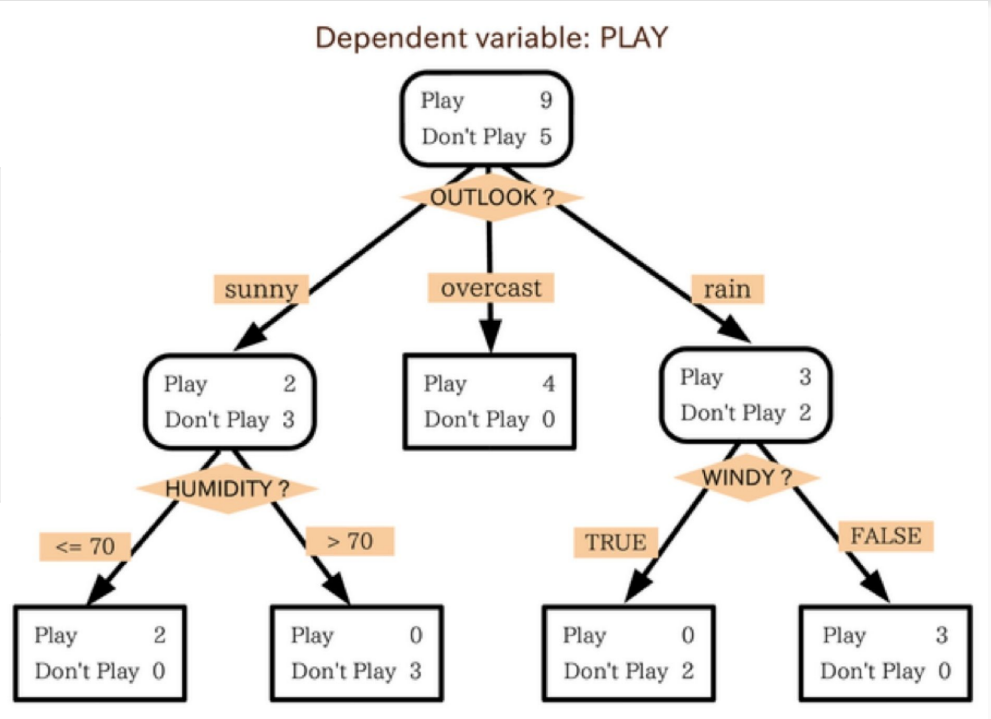
Neural Network





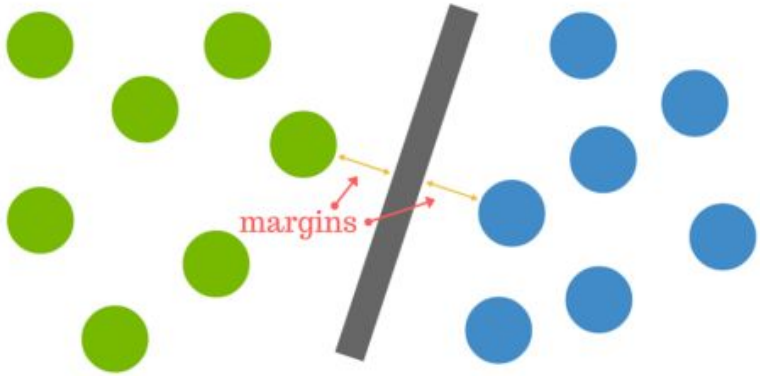
Decision tree

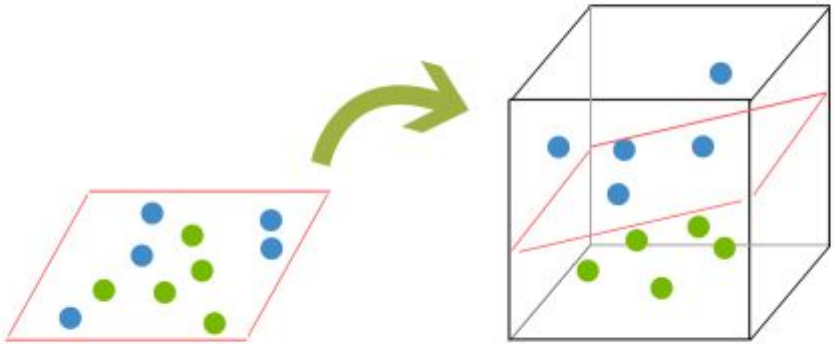
Day	Outlook	Temp.	Humidity	Wind	Play Tennis?
1	Sunny	Hot	High	Weak	No
2	Rain	Cool	Normal	Strong	No
3	*Overcast*	*Hot*	*High*	*Weak*	*Yes*



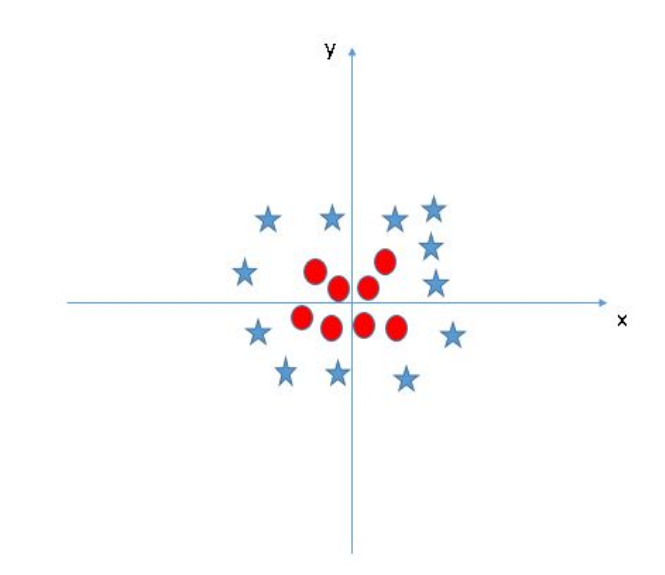
Random forest

SVM (Support Vector Machine)

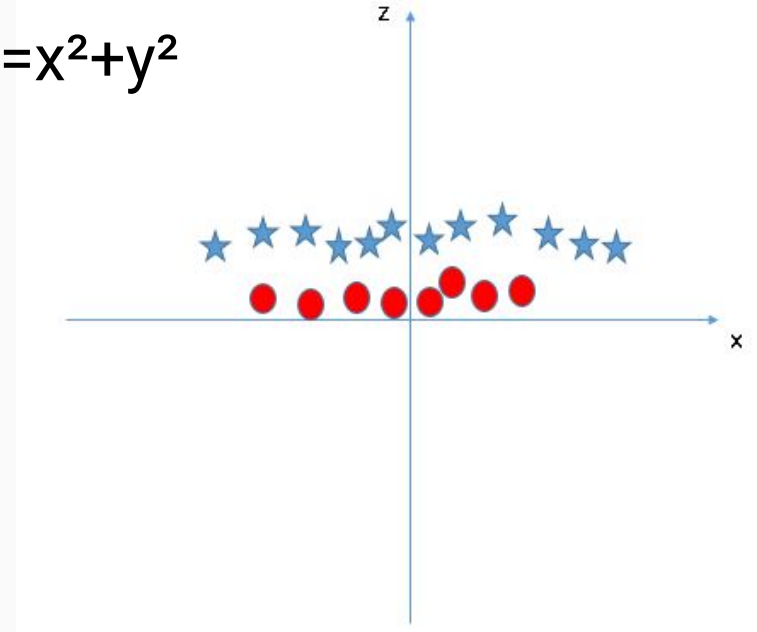




SVM (Support Vector Machine) (2)

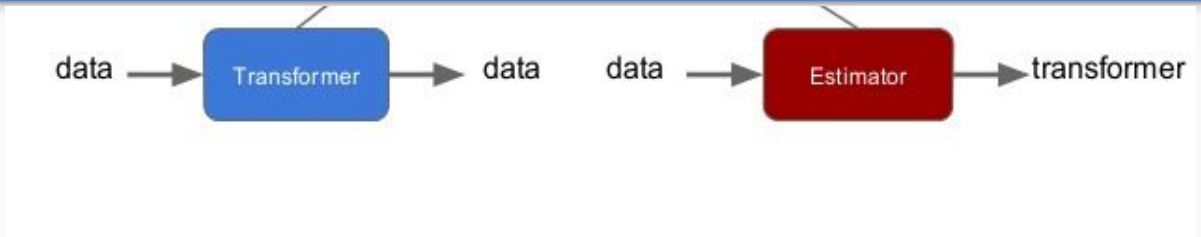


$z = x^2 + y^2$



Spark avancé

Estimator VS Transformer




```
df = spark.read.csv(...)

assembler = VectorAssembler(
    inputCols=[....],
    outputCol="features")

df_output = assembler.transform(df)

df_output.show()
```

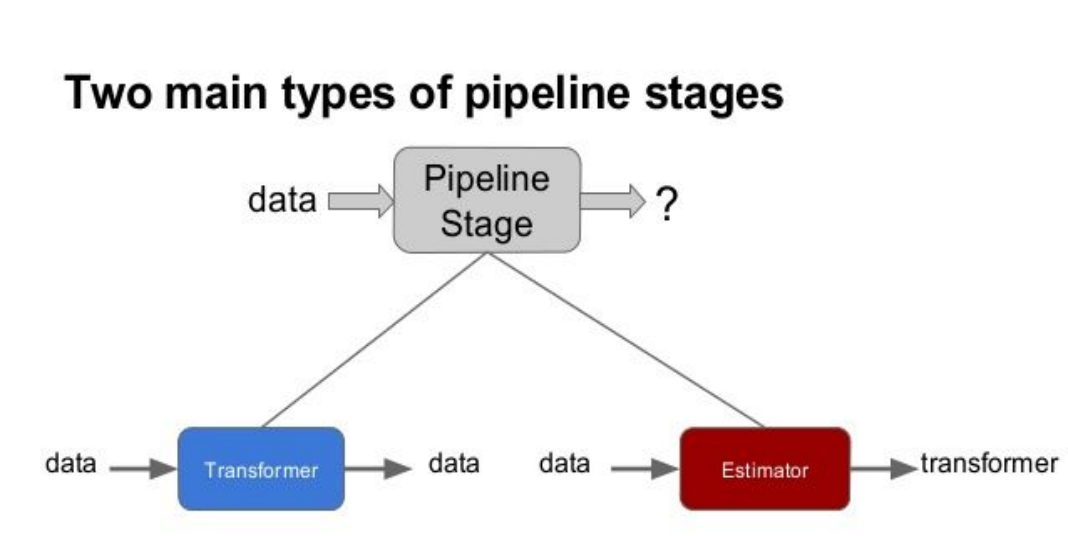
```
df = spark.read.csv(...)

rf = RandomForestClassifier(
    inputCol="features",
    labelCol="label")

model_rf = rf.fit(df)

predictions = model_rf.transform(df)
```

Pipeline




```
# Chain indexer and forest in a Pipeline
pipeline = Pipeline(stages=[featureIndexer, rf])
```

Séance 2

Programme 2ème séance

- Machine-learning : http://spark.apache.org/docs/latest/ml-features.html
  - Attention : ML VS MLlib
  - Transformer
  - Obtenir le meilleur score
    - Ajout de features (col is_pokemon, col number_in_name etc…)
    - Changer les algorithmes
    - Evaluer vos modèles
- Projet text-mining
  - Prise en main données
  - Début du text-mining

Evaluation performances

Precision, recall, f1 score

	p' (Predicted)	n' (Predicted)
p (Actual)	True Positive	False Negative
n (Actual)	False Positive	True Negative

$$precision = \frac{TP}{TP + FP}$$

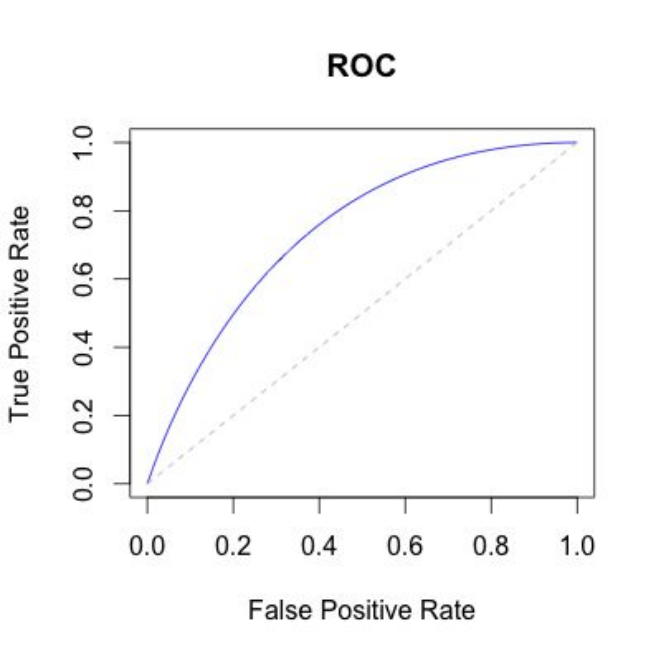
$$recall = \frac{TP}{TP + FN}$$

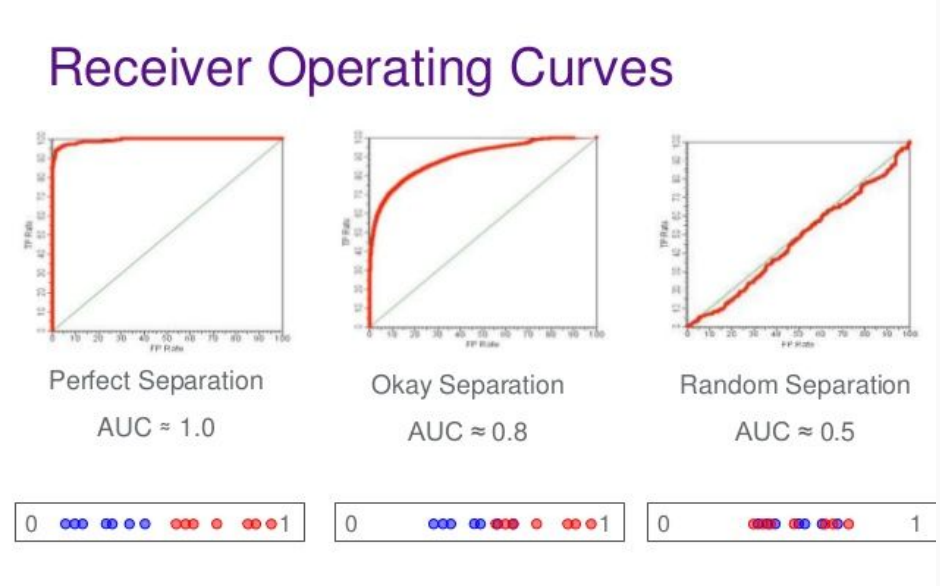
$$F1 = \frac{2 \times precision \times recall}{precision + recall}$$

$$accuracy = \frac{TP + TN}{TP + FN + TN + FP}$$

$$specificity = \frac{TN}{TN + FP}$$

AUC


ROC
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0

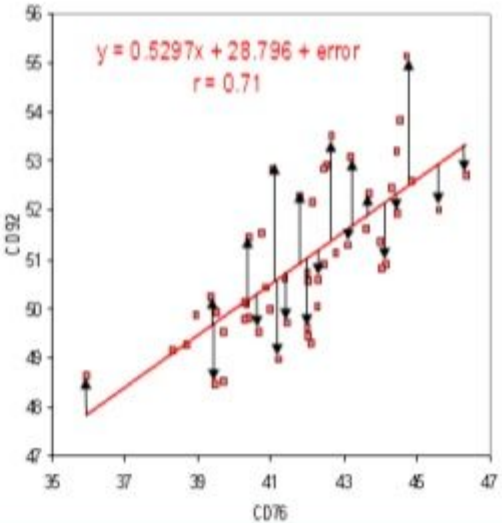

Receiver Operating Curves
Perfect Separation
AUC ≈ 1.0
Okay Separation
AUC ≈ 0.8
Random Separation
AUC ≈ 0.5
0
1

RMSE

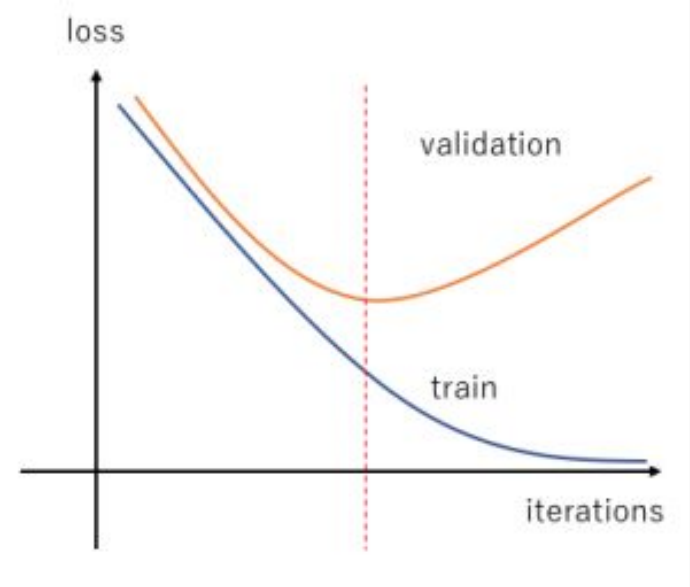
Cost Function - Root Mean squared Error

Way to define how well our model can predict

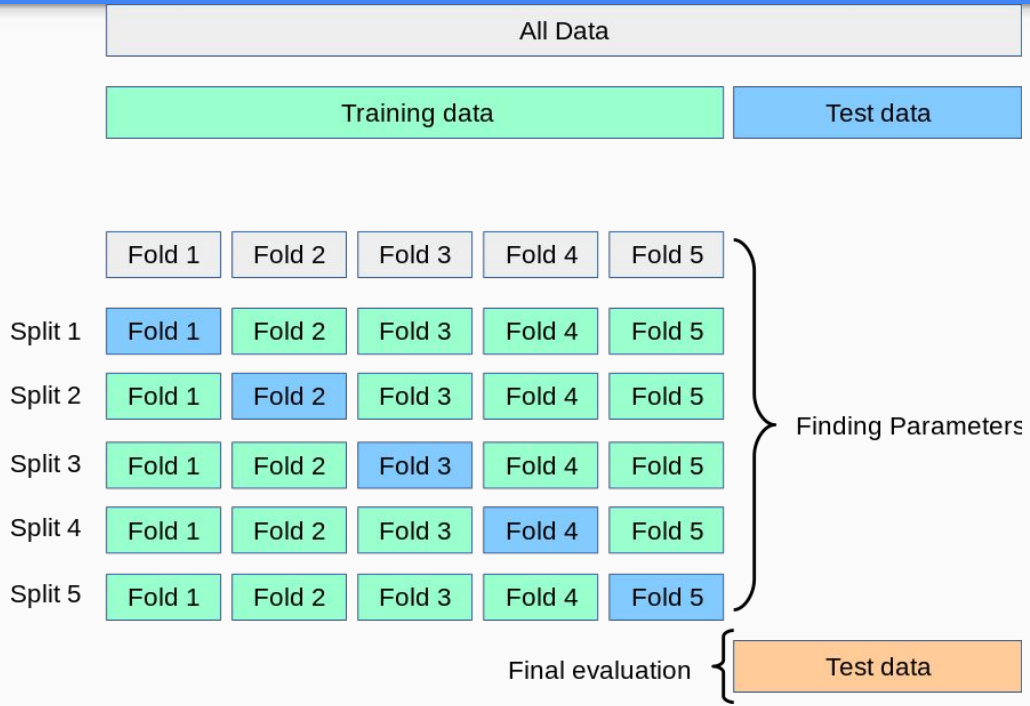
$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$



OverFitting



Cross Validation



Grille de paramètre RandomForest :

- Max Depth : 5,10,20
- NumTree : 10,20,30

Nombre de modèle =
5 (CV) * 3 (Max depth) * 3 (NumTree) = 45 !

Amélioration modèle

- Nettoyage plus intelligent/métier (explicitation des mots, suppression de plus de mots ...)
- Hyper-parameter tuning + cross-validation
- Différents modèles + combinaison de modèles
- Sélection de features
- Ajouts de features (taille de la phrase, taille des mots…)
- Enrichissement des données

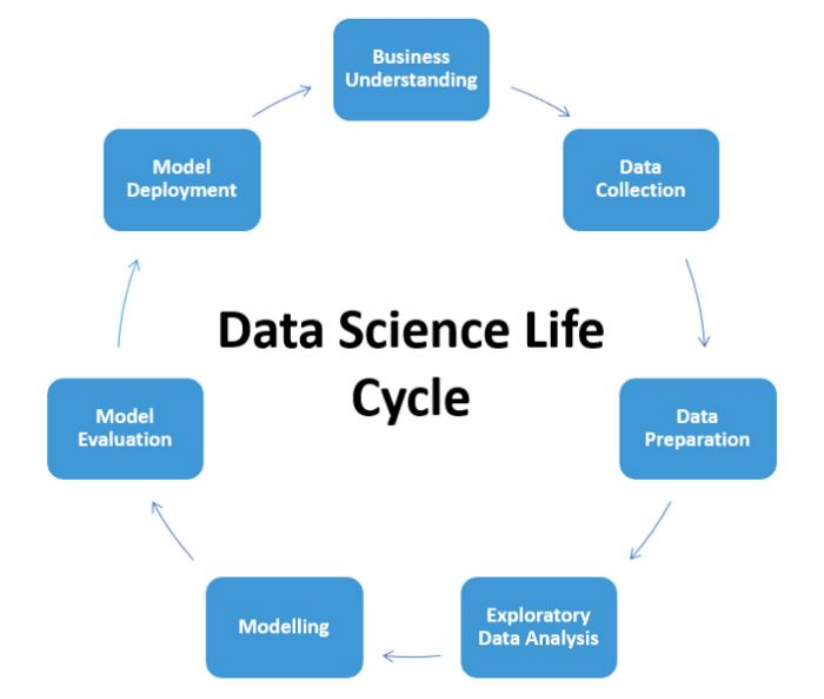
Projet Text-mining

Données projet Text-mining

A	B	C	D	E	F	G	H	I
	id	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
0	22256635	'Nonsense? kiss off, geek. what I said is true. I''ll have your account terminated.'	1	0	0	0	0	0
1	27450690	Please do not vandalize pages, as you did with this edit to W. S. Merwin. If you continu	0	0	0	0	0	0
2	54037174	'" ""Points of interest"" I removed the ""points of interest"" section you added because	0	0	0	0	0	0
3	77493077	'Asking some his nationality is a Racial offence. Wow wasn''t aware of it. Blocking me	0	0	0	0	0	0

	comment_text	toxic
5	'Nonsense? kiss off, geek. what I said is true. I''ll have your account terminated.'	1

Cycle data


Business Understanding
Data Collection
Data Preparation
Exploratory Data Analysis
Modelling
Model Evaluation
Model Deployment
Data Science Life Cycle

Etapes du projet

1 - Nettoyer vos données

2 - Utiliser un algo de vectorisation

3 - Entrainer un algo de ML

4 - Evaluer votre chaîne de traitement

5 - Itérer/améliorer votre chaîne

Objectif projet

- Données :
  - Commentaires toxiques
  - 10 000 lignes, 6 catégories
- Objectifs :
  - Prédire au moins toxic et severe_toxic avec un ou plusieurs modèles
  - Code propre
  - Expliquer et comparer les différentes pipelines testé
  - Avoir un regard critique et lister ce que vous pouvez améliorer
- Rendu individuel :
  - Rapport détaillant vos tests/pipelines de traitement (<10 pages, pas de blabla)
    - Tableaux des scores de vos différentes pipelines (+ nom du paragraphe pour le retrouver dans le notebook)
    - Lister ce qui n’a pas marché
  - 2 notebooks :
    - Un propre avec vos meilleurs expérimentations
    - Un avec vos différents tests

TODO projet

Basique

Avancé

Expert

Pipeline basique

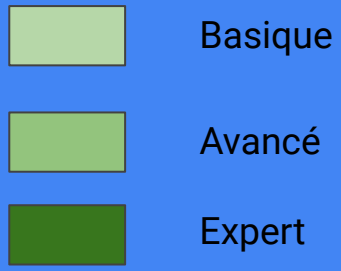
Changer vectorisation

- CV (+ param)
- IDF (+ param)
- W2V (+ param)

Changer modèle

- RF (+param)
- Reg (+param)
- etc…

TODO projet



Pipeline basique

Changer vectorisation

- CV (+ param)
- IDF (+ param)
- W2V (+ param)

Grid Search

Weight / OS

Changer modèle

- RF (+param)
- Reg (+param)
- etc…

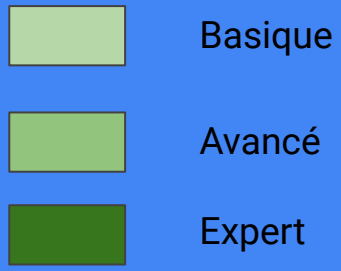
Grid Search

Weight / OS

Analyse résultat

- Analyser les FP
- Analyser les FN

TODO projet



Pipeline basique

Changer vectorisation

- CV (+ param)
- IDF (+ param)
- W2V (+ param)

Grid Search

Weight / OS

Changer modèle

- RF (+param)
- Reg (+param)
- etc…

Grid Search

Weight / OS

Analyse résultat

- Analyser les FP
- Analyser les FN

- Analyser voc par categorie

- Changer le nettoyage
- Correction orthographique (peu prendre du temps)
- Faire des collocation/correspondance …
- Corriger des labels

Conseils projets

- Bien s’organiser :
  - Tenir un excel avec les perfs en fonction des paramètres
  - Faire plusieurs notebooks
  - nettoyer vos notebooks
- Tester différentes choses :
  - Analyse du vocabulaire
  - plusieurs modèles/words embedding
  - Différent nettoyage
  - Correcteur orthographique
  - Analyse des erreurs pour améliorer la pipeline Cross validation
  - Prédire les différentes classes
  - Fonction “démo” qui teste sur une phrase

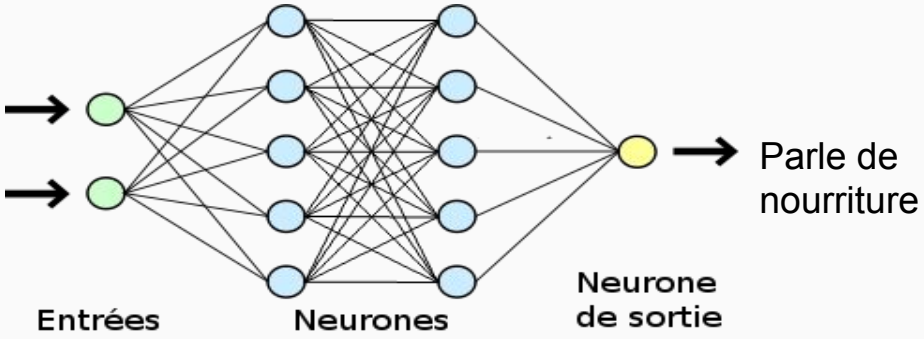
Text-mining nettoyage

Objectifs

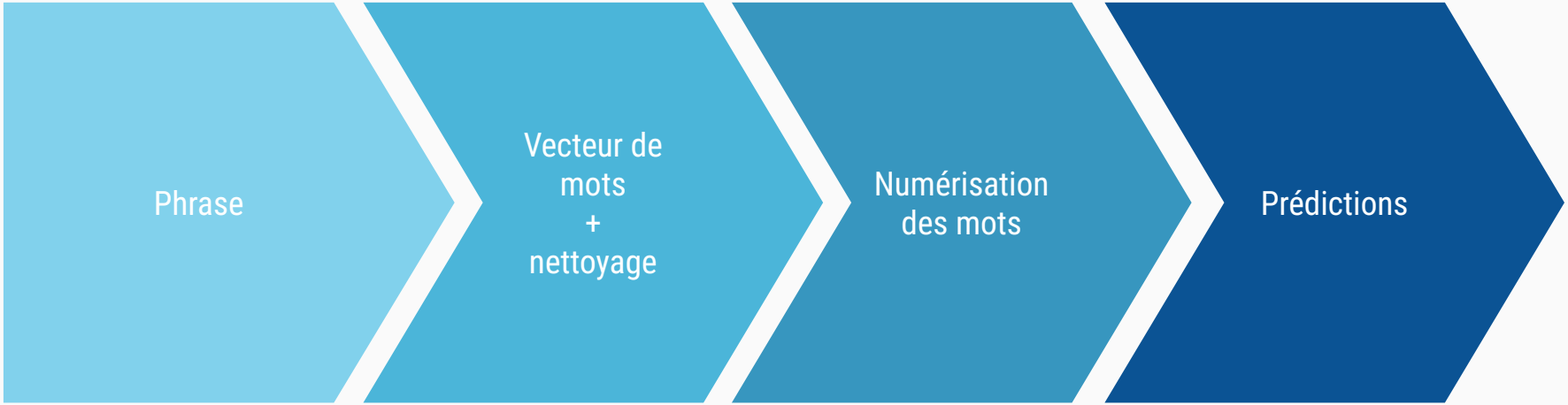
C’est le moment de manger les harikots

?

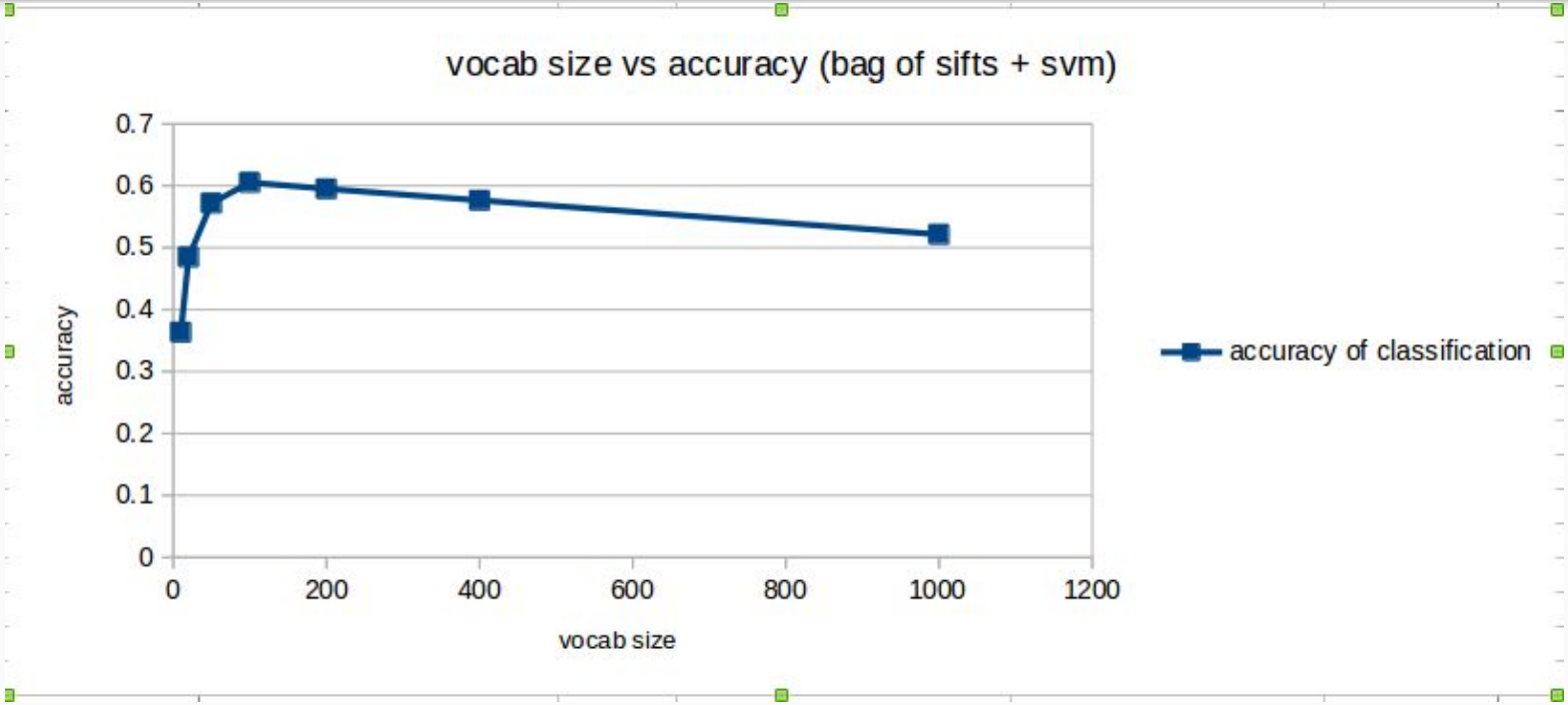
[2;0.46]



Pipeline


Phrase
Vecteur de mots + nettoyage
Numérisation des mots
Prédictions

Réduire la taille du vocabulaire



Nettoyage de texte

<u>Lemmatization :</u> transformer un mot en son radical (lemme)

<u>Exemple :</u>

Chevaux → Cheval
Chevauchez → Chevaucher

<u>Stemming:</u> supprimer la fin des mots

<u>Exemple :</u>

Chevaux → Cheva
Chevauchez → Cheva

<u>Collocation:</u> lier des mots qui apparaissent souvent ensemble

<u>Exemple :</u>

"New-York" + "times"
→ "New-York Times"

<u>Part of speech tagging :</u> rajouter des précision sur le mot

<u>Exemple :</u>

Chevaux → Chevaux/NOM
Chevaucher → Chevaucher/VER:infi

Exemple de code NLTK (1)

Part of speech tagging :

```
In [11]: text= nltk.word_tokenize("Hello, I like hot chocolate")
         pos_tag=nltk.pos_tag(text)
         pos_tag

Out[11]: [('Hello', 'NNP'),
          (',', ','),
          ('I', 'PRP'),
          ('like', 'VBP'),
          ('hot', 'JJ'),
          ('chocolate', 'NN')]
```

Stemming :

```
In [6]: from nltk.stem.lancaster import LancasterStemmer
        st = LancasterStemmer()
        st.stem('cats')

Out[6]: 'cat'
```

Exemple de code NLTK (2)

<u>Lemmatization :</u>

Noun	n
Adj	a
Verb	v
Adv	r

```
In [10]: from nltk.stem import WordNetLemmatizer
         wordnet_lemmatizer = WordNetLemmatizer()
         print(wordnet_lemmatizer.lemmatize("dogs"))
         print(wordnet_lemmatizer.lemmatize("are"))
         print(wordnet_lemmatizer.lemmatize("are", pos="v"))

         dog
         are
         be
```

Exemple de code Spacy

```
doc = nlp(u'Apple is looking at buying U.K. startup for $1 billion')

for token in doc:
    print(token.text, token.lemma_, token.pos_, token.tag_, token.dep_,
          token.shape_, token.is_alpha, token.is_stop)
```

TEXT	LEMMA	POS	TAG	DEP	SHAPE	ALPHA	STOP
Apple	apple	PROPN	NNP	nsubj	Xxxxx	True	False
is	be	VERB	VBZ	aux	xx	True	True
looking	look	VERB	VBG	ROOT	xxxx	True	False

Exemple (1)

C’est le moment de manger les harikots.

Tokenisation

[C ; est ; le ; moment ; de ; manger ; les ; harikots]

Correction orthographique

[C ; est ; le ; moment ; de ; manger ; les ; haricots]

Exemple (2)

[C ; est ; le ; moment ; de ; manger ; les ; haricots]

Lemmatization

Stemming

[C ; être ; le ; moment ; de ; manger ; les ; haricot]

[C ; est ; le ; mom ; de ; mange ; le ; haricot]

Exemple (3)

[C ; être ; le ; moment ; de ; manger ; les ; haricot]

Suppression des petits mots

[être ; moment ; manger ; haricot]

Suppression des Stops Words

[moment ; manger ; haricot]

Word2Vec, TF-IDF….

Première fonction spacy

- après avoir pris en main spacy faire une fonction qui :
  - Lemmatize
  - retire les stops words
  - garde les mots de plus de 2 caractères
  - met tout en minuscule
- Etape 2 : passer votre fonction en udf sur votre dataframe
  - verifier que Toxic ne contient bien que 0 ou 1
  - utiliser df.limit(100) pour appliquer votre transformation sur un echantillon

Pos-tagging

- Texte original : *Nous sommes allées en Bretagne contempler de magnifiques allées couvertes du Néolithique.*
- Texte étiqueté : *Nous/PRO:PER sommes/VER:pres allées/VER:pper en/PRP/en Bretagne/NAM contempler/VER:infi de/PRP magnifiques/ADJ allées/NOM couvertes/VER:pper du/PRP:det Néolithique/NAM ./SENT*

Caractéristiques sur le texte

- Caractéristique sur la ponctuation
- Nombre de mots
- Nombre de caractères par mots
- Nombre de verbe
- ...

“Règle de l’oignon”

Je voudrais un oignon

oignon jaune ?
oignon rouge ?
oignon blanc ?

Je voudrais un oignon jaune

Text-mining avancé

Approches

<u>Approche simple :</u>
Vocabulary: {‘shining’ : 0, 'sun': 1, 'bright': 2, 'sky': 3}

"The sun in the sky is bright." → [0 1 1 1]
"We can see the shining sun, the bright sun." → [1 2 1 0]

<u>Words embedding :</u>

Rouge : [1;0;0]	Orange : [1;0;1]
Bleu : [0;1;0]	Blanc : [1;1;1]
Jaune : [0;0;1]	Violet : [1;1;0]

TF-IDF (1)

Term Frequency-Inverse Document Frequency

Principe : Les mots souvent présent dans les documents ont un poids faible

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

TF-IDF (2)

Exemple :

Vocabulary: {‘shining’ : 0, 'sun': 1, 'bright': 2, 'sky': 3}

"The sun in the sky is bright." → [0 1 1 1]
"We can see the shining sun, the bright sun." → [1 2 1 0]

IDF : [0.30, 0, 0, 0.30]

"The sun in the sky is bright." → [0 0 0 0.3]
"We can see the shining sun, the bright sun." → [0.3 0 0 0]

Word2Vec (1)

[Paris ; grand ; ville]

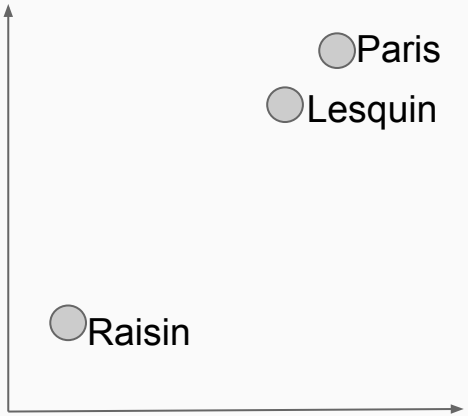
[Paris ; ville ; france]

[Lesquin ; petit ; ville]

[Lesquin ; ville ; france]

[Raisin ; petit ; fruit]

Principe : Rapprocher les mots utilisés dans le même contexte



Word2Vec (2)

Source Text

The quick brown fox jumps over the lazy dog. ⟹

The quick brown fox jumps over the lazy dog. ⟹

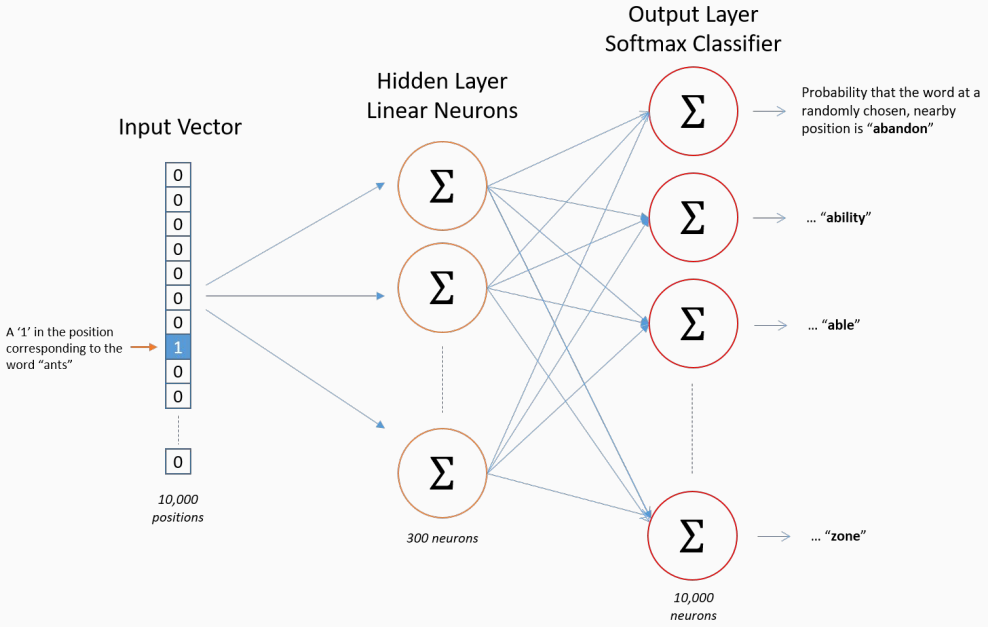
The quick brown fox jumps over the lazy dog. ⟹

Training Samples

(the, quick)
(the, brown)

(quick, the)
(quick, brown)
(quick, fox)

(brown, the)
(brown, quick)
(brown, fox)
(brown, jumps)



"the" va avec "quick" dans 5% des cas
"the" va avec "brown" dans 2% des cas
....

Classe déséquilibré

Attention aux métriques !

- Si j'ai 99% de négatif et 1% de positif
- Si je prédit tout en négatif sauf 1 positif bien prédit alors :
  - precision sur positif = 1/1+0 =100%
  - precision sur negatif = 99%

Attention aux weighted metrics (weighted = moyenne pondéré entre la classe positive et la classe négative)

Affichez une matrice de confusion pour vous rendre compte

$$precision = \frac{TP}{TP + FP}$$

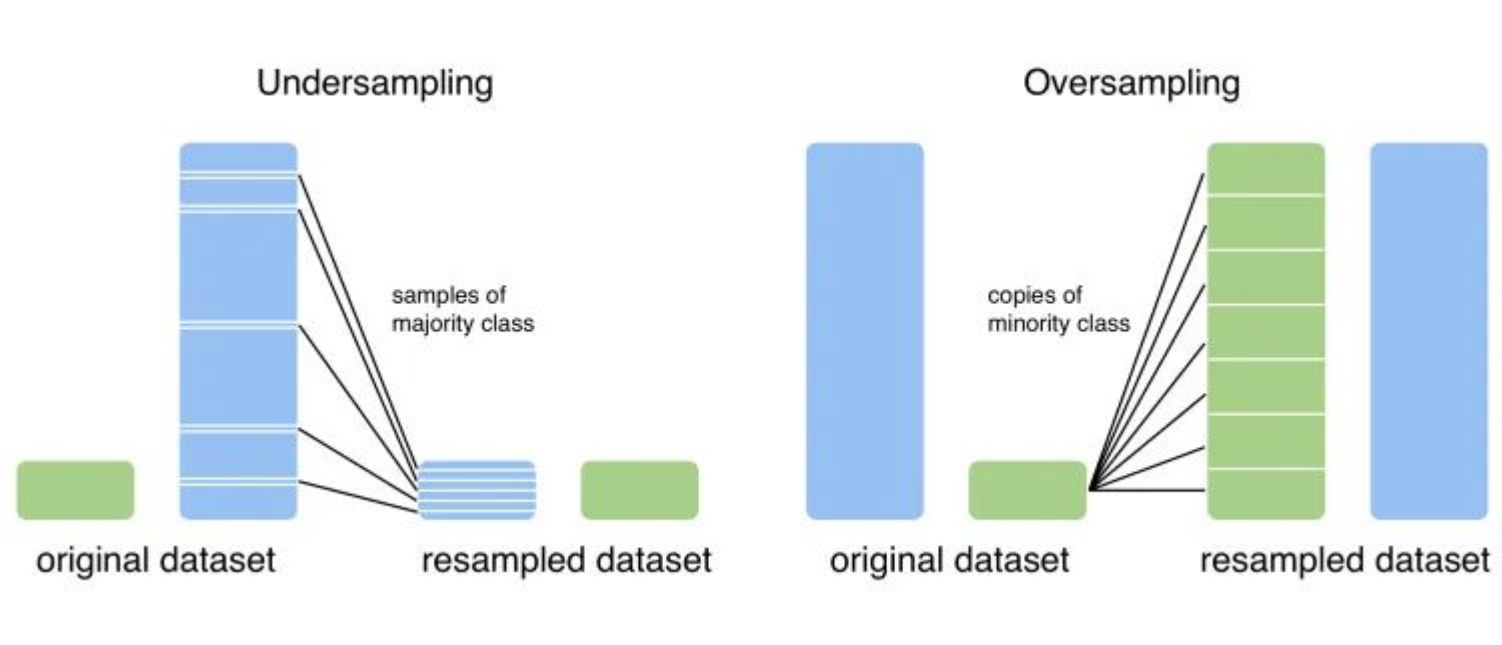
$$recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times precision \times recall}{precision + recall}$$

$$accuracy = \frac{TP + TN}{TP + FN + TN + FP}$$

$$specificity = \frac{TN}{TN + FP}$$

Under/over sampling



Influence sur les poids

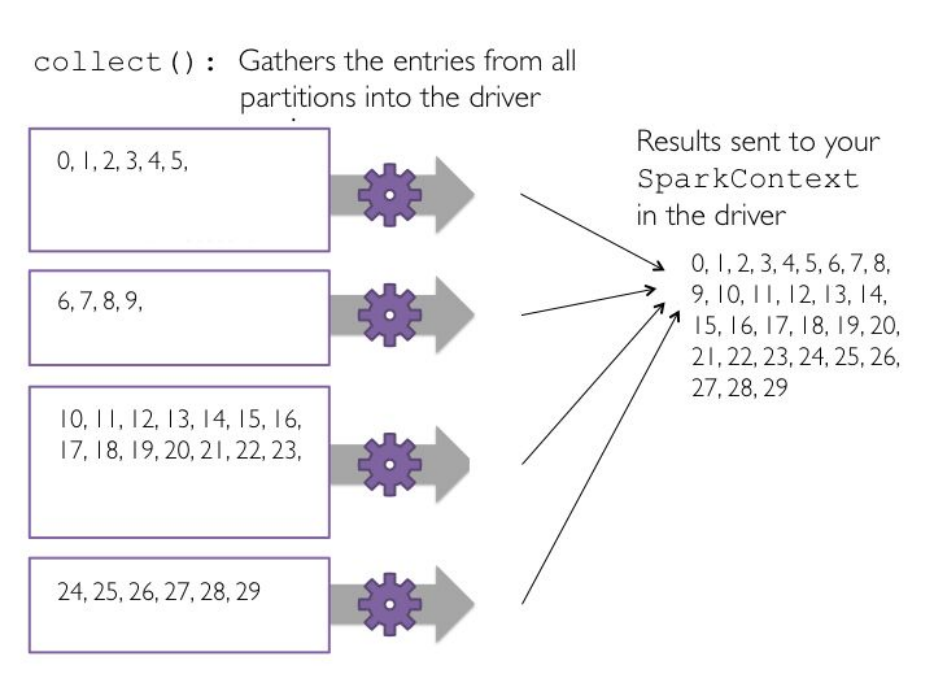
- Changer les poids permet d'impacter l'entraînement du modèle
- Si le modèle se trompe sur un toxic c'est comme se tromper sur 15 non toxic

id	comment_text	clean	toxic	prediction	weight
22256635	'Nonsense? kiss ...	nonsense kiss gee...	1	0.0	15
150009866	'"Well I just fin...	finish good bit e...	0	0.0	1
103100000873	Please refrain fr...	refrain add nonse...	0	0.0	1
103121885338	'What?Then stop d...	what?then stop de...	0	0.0	1
103209224017	Some advice An ar...	advice article ma...	0	0.0	1
103210665191	Thanks for experi...	thank experiment ...	0	0.0	1

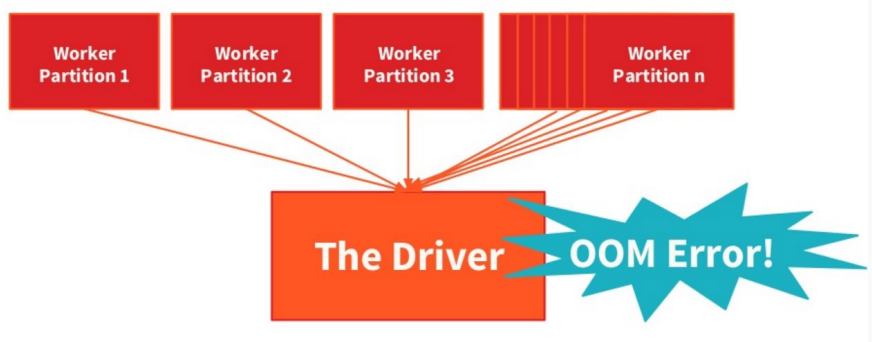
Dernière séance

Optimisation Spark

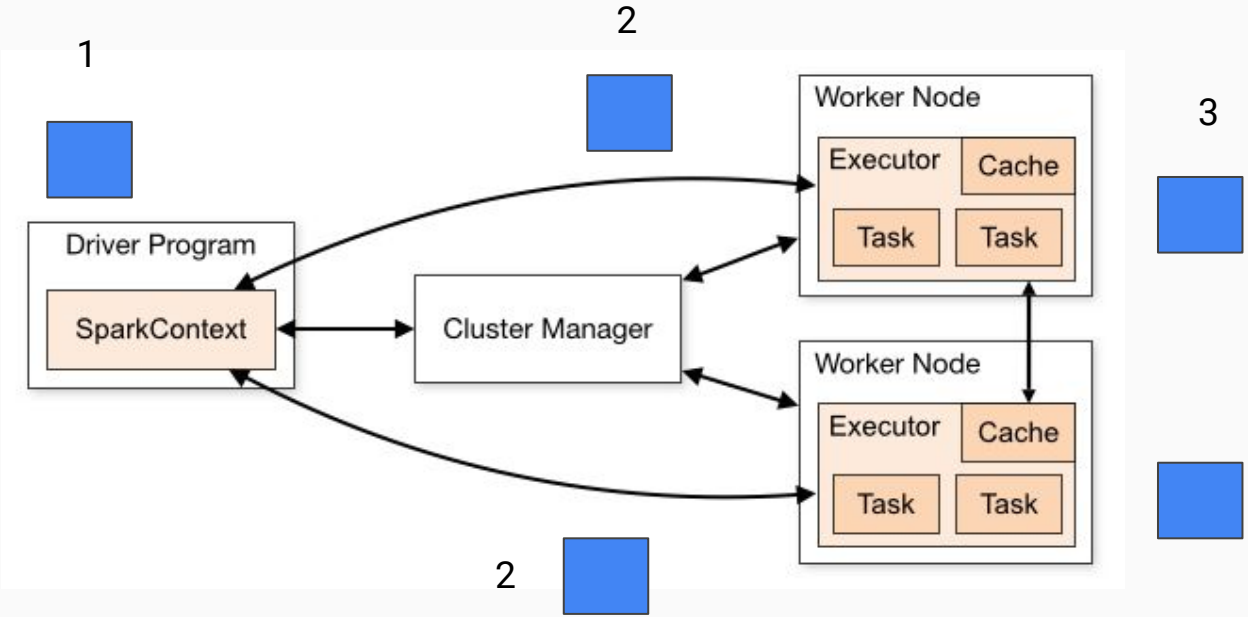
Collect en Spark



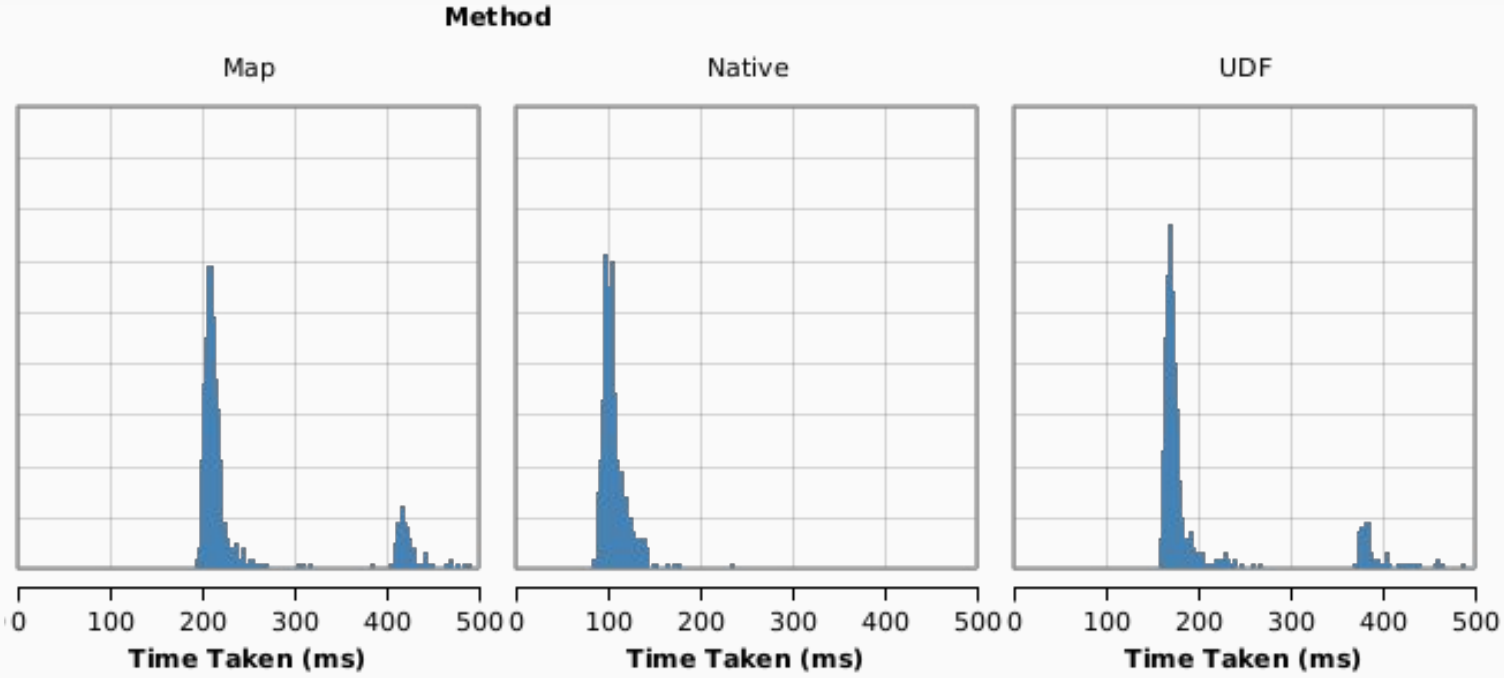
Bien réflechir avant de le faire !



Context et variables envoyés aux workers

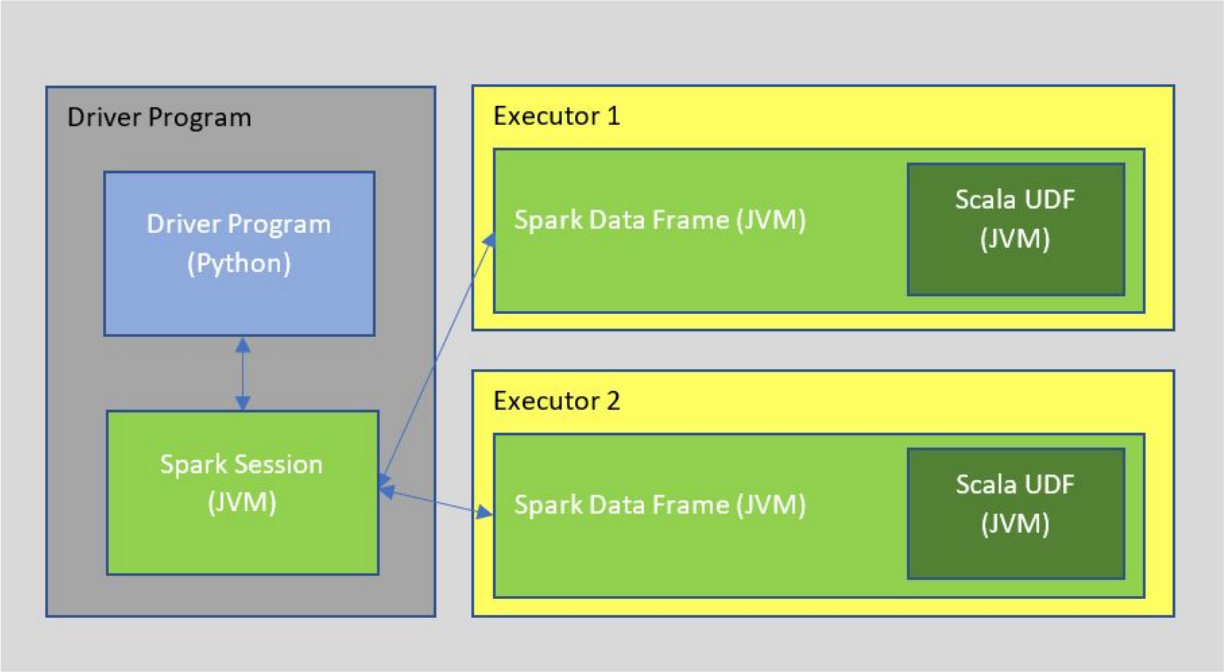


UDF VS Map VS Native



Native

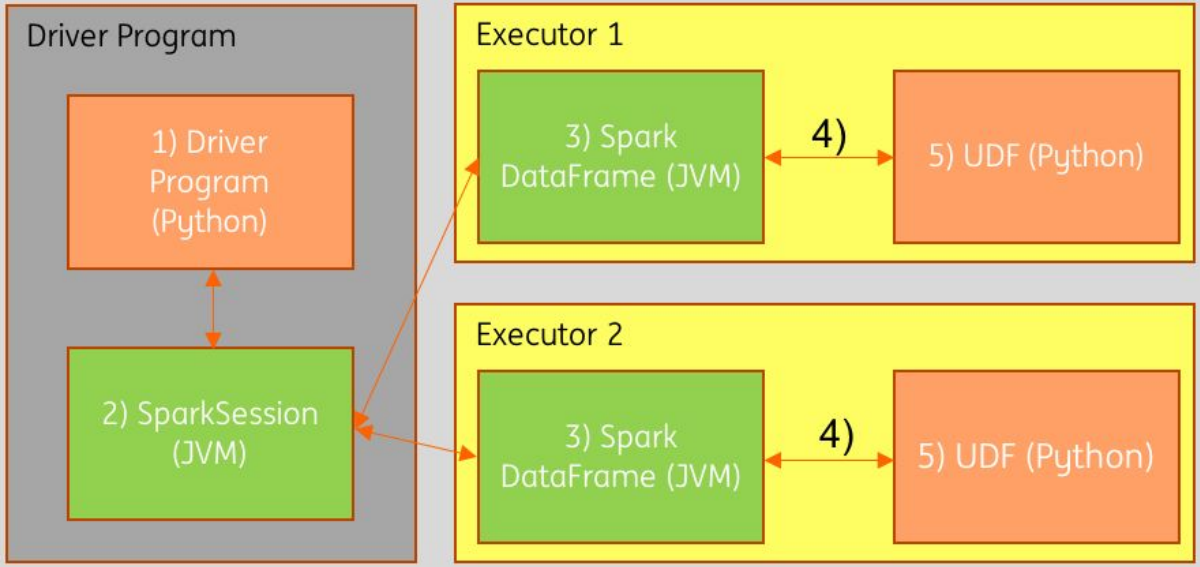
- Spark codé en Scala
- Le code ne quitte pas la JVM
- Les données ne quittent pas la JVM



UDF python

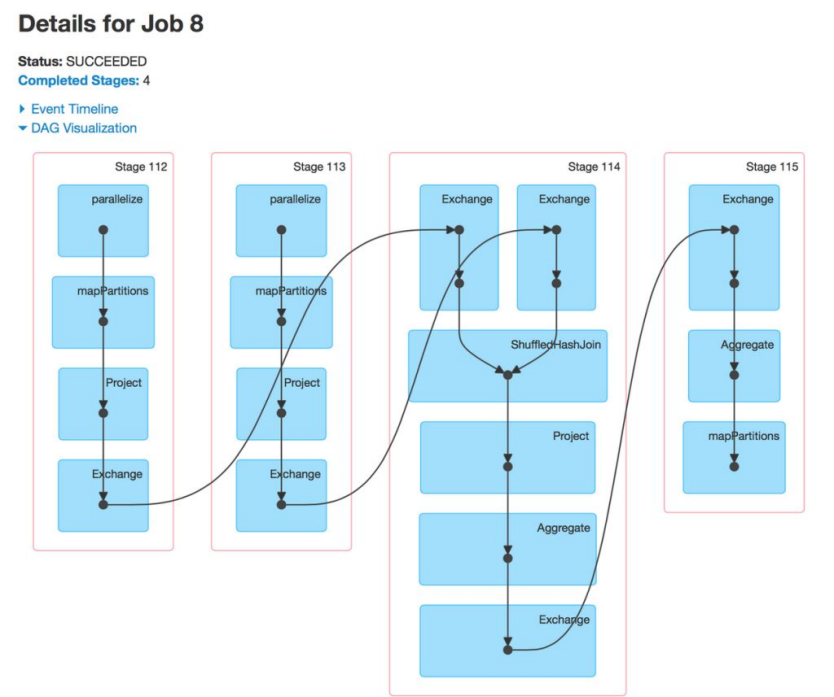
- Spark codé en Scala

- 1 - Le code python est envoyé à la JVM
- 2 - La JVM envoie les informations aux executor
- 3 - Chargement des données dans la JVM + operation dans la JVM
- **4 - Sérialisation + Désérialisation**
- 5 - Execution code python
- **4 - Sérialisation + Désérialisation**



DAG Spark

- Spark est lazy, le code ne s'exécute que lorsque c'est nécessaire
- A chaque opération Spark complète le DAG
- Opération décrite à bas niveau (map et reduce)
- Permet de consulter l'avancement des jobs



Format Parquet

- Compression des données
- Conserve des méta-données
- Format binaire
- Format orienté colonnes

Dataset	Size on Amazon S3	Query Run time
Data stored as CSV files	1 TB	236 seconds
Data stored in Apache Parquet format*	130 GB	6.78 seconds
Savings / Speedup	87% less with Parquet	34x faster

Columns Database VS Row Database

Tabular Data

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

Column-oriented Layout

A1	A2	A3	B1	B2	B3	C1	C2	C3

Row-based Layout

A1	B1	C1	A2	B2	C2	A3	B3	C3

- Column = efficace pour requêter
- Row = efficace pour ajouter des lignes

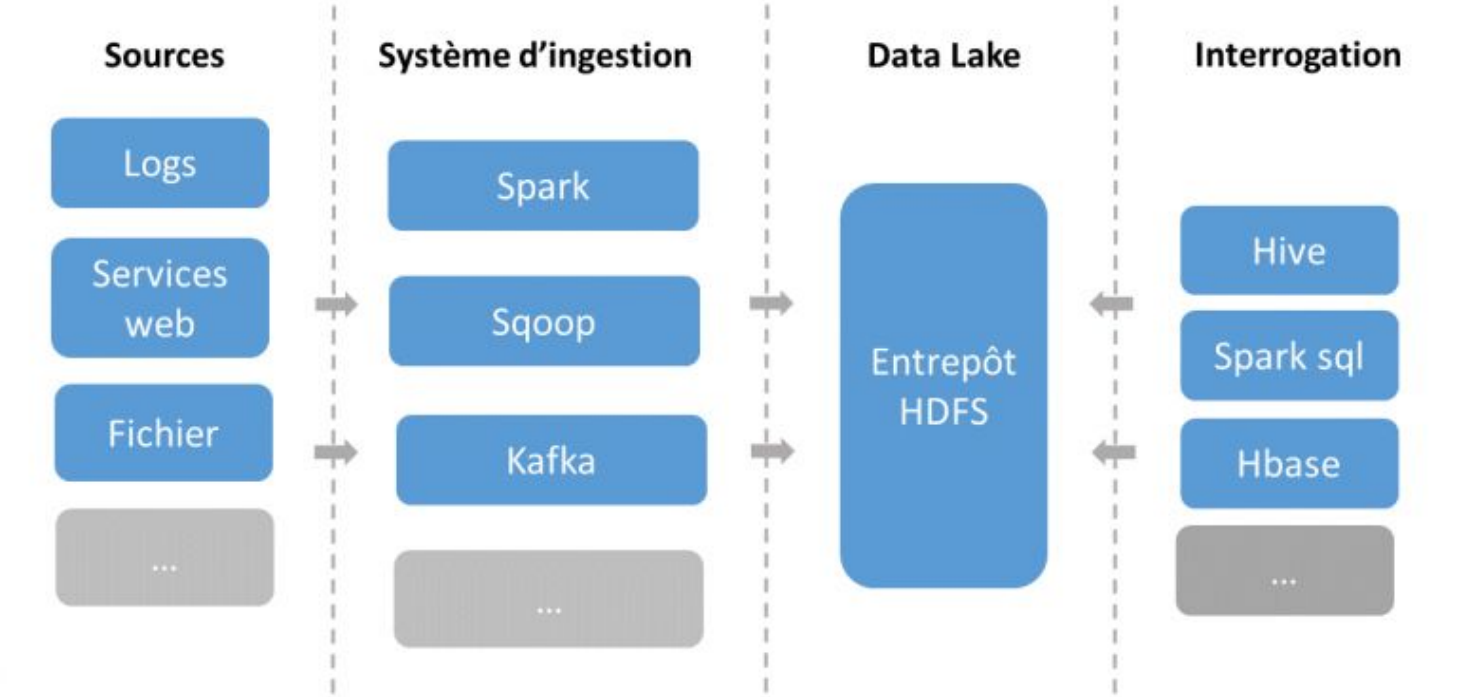
Architecture Big Data

Besoins de votre plateforme

- Stockage Data-lake
  - Stockage multi-données
- Traitement temps réel
  - Réponse rapide
- Traitement sur le Data-lake
  - Traitement de large volume de données froides
- Requêtage

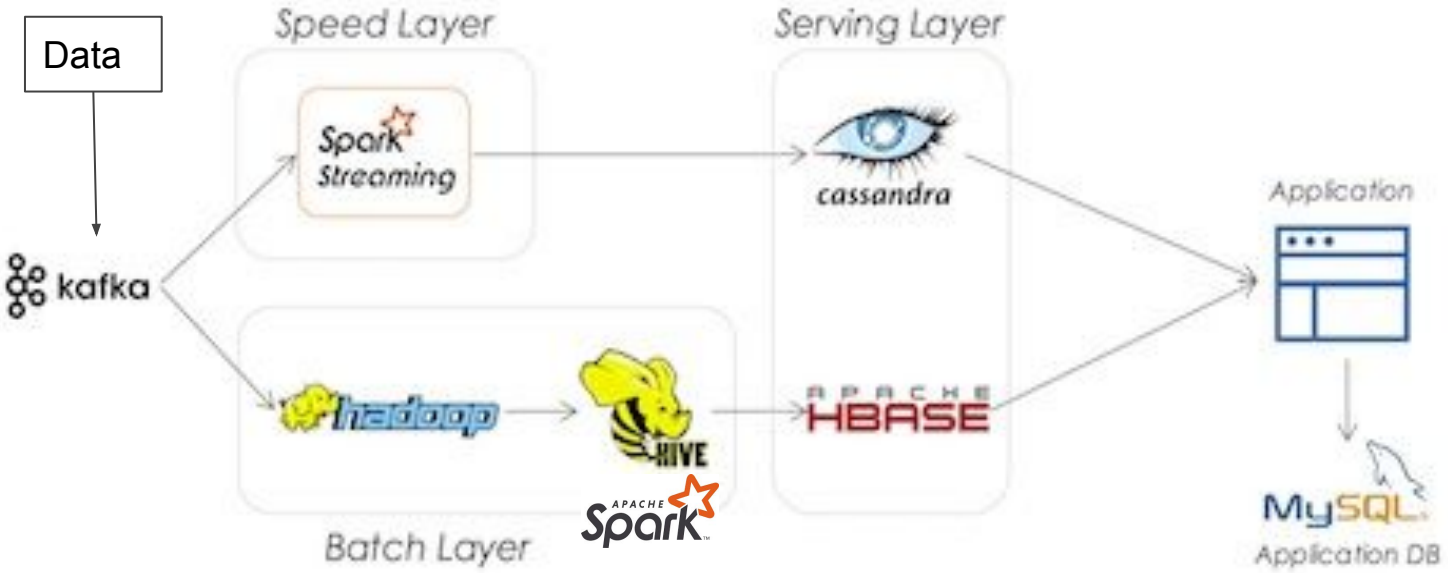
Datalake seul

- Stockage multi source
- Requêtage
- Architecture basique



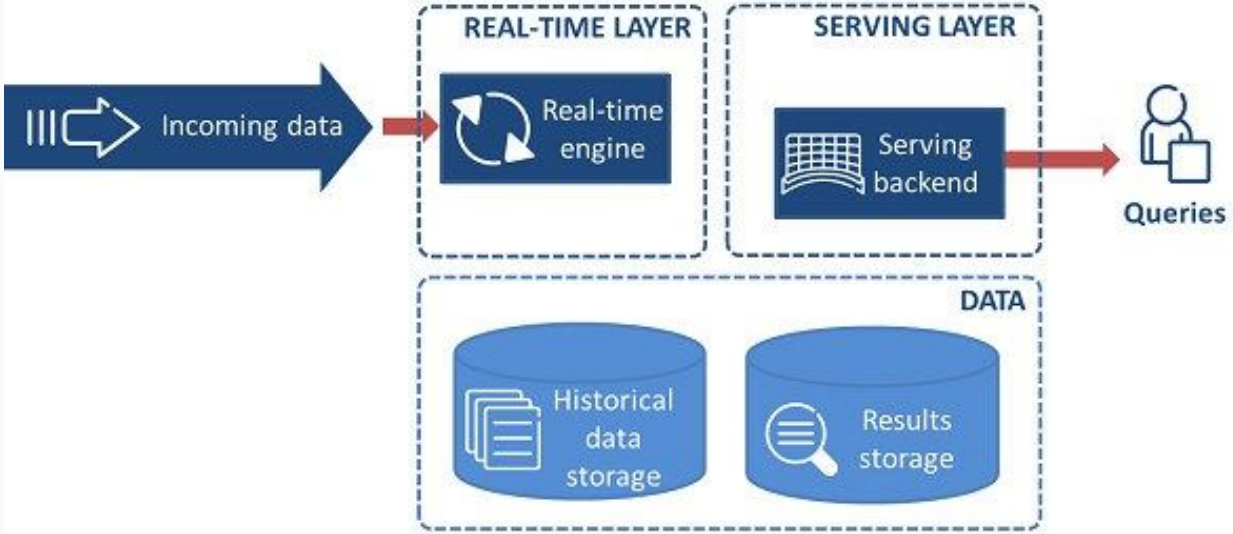
Lambda

- Couche temps réel
- Couche de traitement
- Couche de requêtage
- Complexe



Kappa

- Fusion temps réel et batch
- Utile quand le code entre les deux est identique
- Simplifie l’architecture

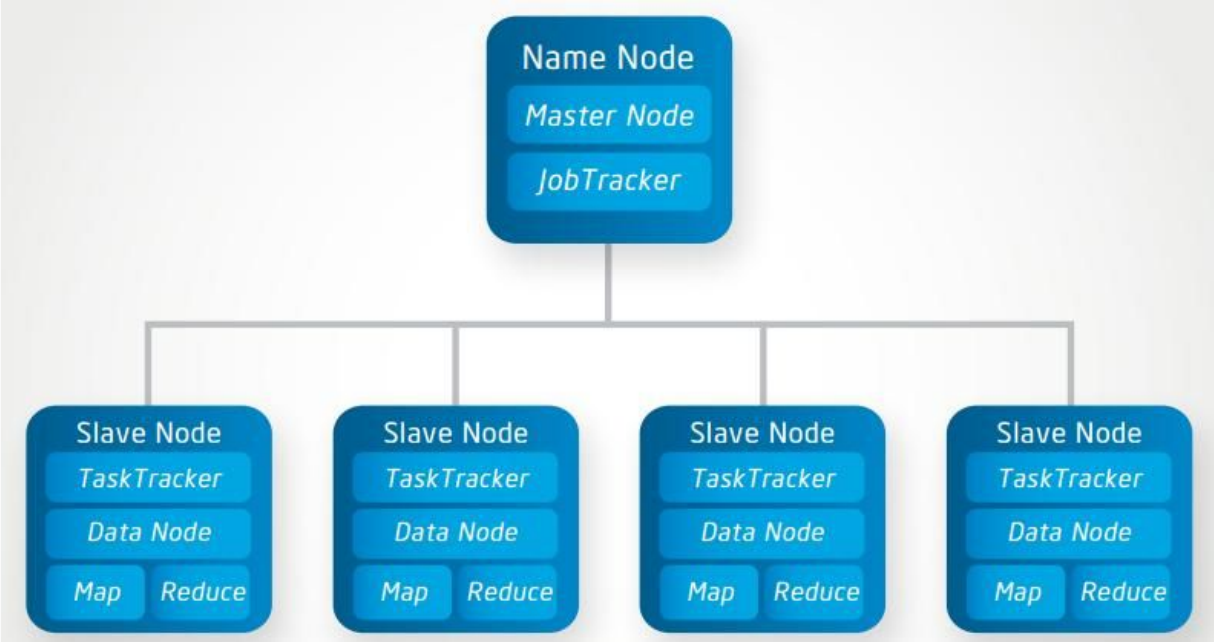


Installer son cluster

- Plusieurs machines (Linux)
- Java
- Droits d’administrateurs
- Connection SSH
- Stockage sur les machines
- RAM sur les machines

Déployer Hadoop

- Copier le logiciel sur toutes les machines
- Donner les rôles entre les machines
- Autoriser les connexion entre les machines



Configurer Hadoop

- Configurations for NameNode:

Parameter	Value
dfs.namenode.name.dir	Path on the local filesystem where the NameNode stores the namespace and transactions logs persistently.
dfs.hosts / dfs.hosts.exclude	List of permitted/excluded DataNodes.
dfs.blocksize	268435456
dfs.namenode.handler.count	100

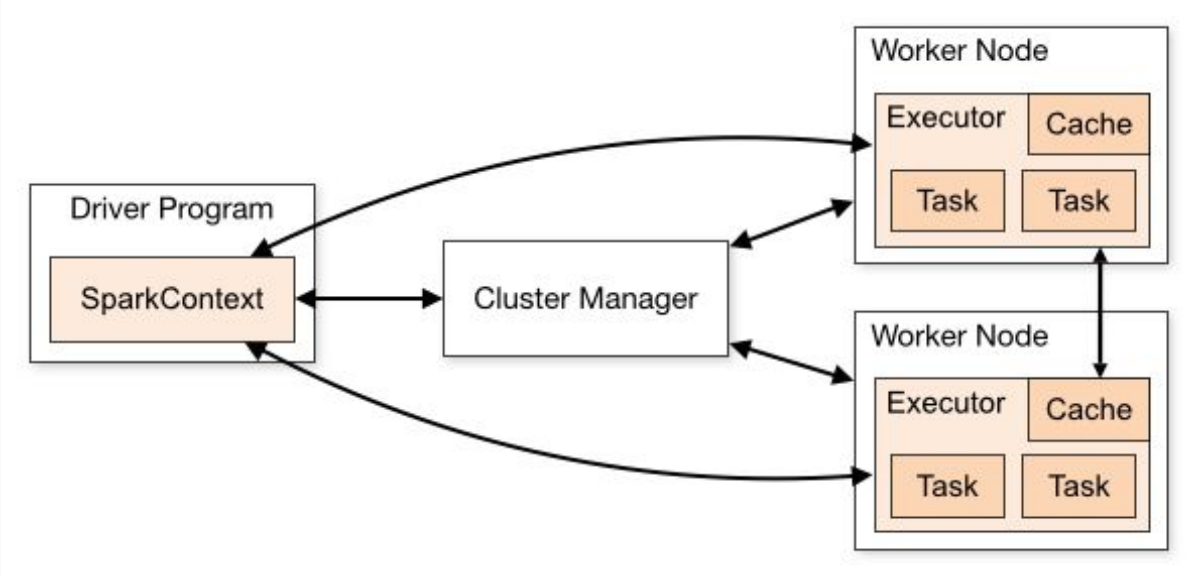
- Configurations for DataNode:

Parameter	Value	Notes
dfs.datanode.data.dir	Comma separated list of paths on the local filesystem of a DataNode where it should store its blocks.	If this is different

https://hadoop.apache.org/docs/r2.4.1/hadoop-project-dist/hadoop-hdfs/hdfs-default.xml

Déployer Spark

- Copier le logiciel sur toutes les machines
- Donner les rôles entre les machines
- Autoriser les connexion entre les machines
- Système de stockage partagé (HDFS, NFS, ….)



Configurer Spark

https://spark.apache.org/docs/latest/configuration.html

Property Name	Default	Meaning
`spark.app.name`	(none)	The name of your application. This will appear in the UI and in log data.
`spark.driver.cores`	1	Number of cores to use for the driver process, only in cluster mode.
`spark.driver.maxResultSize`	1g	Limit of total size of serialized results of all partitions for each Spark action (e.g. collect) in bytes. Should be at least 1M, or 0 for unlimited. Jobs will be aborted if the total size is above this limit. Having a high limit may cause out-of-memory errors in driver (depends on spark.driver.memory and memory overhead of objects in JVM). Setting a proper limit can protect the driver from out-of-memory errors.
`spark.driver.memory`	1g	Amount of memory to use for the driver process, i.e. where SparkContext is initialized, in the same format as JVM memory strings with a size unit suffix ("k", "m", "g" or "t") (e.g. `512m`, `2g`). *Note:* In client mode, this config must not be set through the `SparkConf` directly in your application, because the driver JVM has already started at that point. Instead, please set this through the `--driver-memory` command line option or in your default properties file.
`spark.driver.memoryOverhead`	driverMemory * 0.10, with minimum of 384	The amount of off-heap memory to be allocated per driver in cluster mode, in MiB unless otherwise specified. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the container size (typically 6-10%). This option is currently supported on YARN and Kubernetes.
`spark.executor.memory`	1g	Amount of memory to use per executor process, in the same format as JVM memory strings with a size unit suffix ("k", "m", "g" or "t") (e.g. `512m`, `2g`).

Installer son cluster en production

- Déploiement multi-machines de conf/ Spark/ Hadoop/ autres
- Monitoring des packages sur le cluster
- Monitoring des droits sur Hadoop (Kerberos)
- Monitoring des ressources (Yarn, Mesos, …)
- Système d’alerting d’usage des ressources (RAM, core, stockage)
- Procédure d’ajout/suppression de machines (data balancing)
- Upgrade des versions
- ...

Avoir son cluster

	Interne	SAS	Cloud public
+	- Contrôle total de la plateforme - Coût sur le long terme faible	- Simple d'utilisation - Support	- Coût de mise en place faible - Scalable
-	- Compétence interne élevé - Coût d'installation élevé	- Développement spécifique coûteux - Usage spécifique	- Chère pour la production - Nécessite des compétences pour la mise en place

Pour aller plus loin

Autres words embedding

fastText

CamemBERT

OpenAI

GPT-3, an autoregressive language model with 175 billion parameters

https://beta.openai.com/playground

Bert

1 - Semi-supervised training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.

Semi-supervised Learning Step

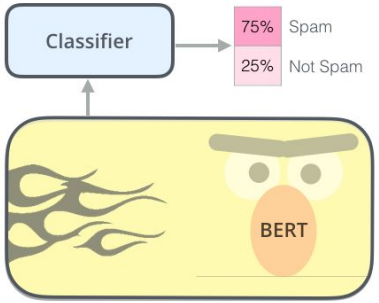
Model:

Dataset:

Objective: Predict the masked word (langauge modeling)

2 - Supervised training on a specific task with a labeled dataset.

Supervised Learning Step



Model: (pre-trained in step #1)

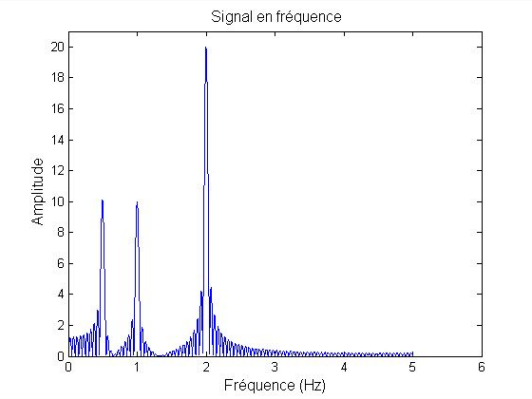
Dataset:

Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached…	Not Spam

Images

$$\begin{pmatrix} 16 & 3 & 2 & 13 \\ 5 & 10 & 11 & 8 \\ 9 & 6 & 7 & 12 \\ 4 & 15 & 14 & 1 \end{pmatrix}$$

Son


Signal en fréquence
Amplitude
Fréquence (Hz)
0
2
4
6
8
10
12
14
16
18
20
0
1
2
3
4
5
6

Liens utile

Spark

- Spark : https://spark.apache.org/
- Doc Pyspark : http://spark.apache.org/docs/2.2.0/api/python/index.html
- Doc ML : https://spark.apache.org/docs/latest/ml-guide.html